Claude Certified Architect — Foundations Certification

مطالعہ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

Claude Certified Architect — Foundations سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر حقیقی دنیا کے Claude-based وقت درست حل نافذ کرتا ہے یہ امتحان trade-off فیصلہ کر سکتا ہے۔ Claude Code، Model Context Protocol (MCP) یعنی — یہ کہ بنیادی علم کا جائزہ لیتا ہے Claude Agent SDK، Claude API، اور کے ساتھ پروڈکشن ایپلیکیشنز بنائی جاتی ہیں Claude وہ بنیادی ٹیکنالوجیز جن سے

امتحانی سوالات حقیقت پسندانہ صنعتی منظرناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے agentic systems بنانا، multi-agent research pipelines ڈیزائن کرنا، Claude Code کو CI/CD میں ضم کرنا، developer productivity tools بنانا، اور غیر ساختہ دستاویزات سے structured data نکالنا

ہدف امیدوار

مثالی امیدوار ایک **solution architect** جو ہے Claude اور پروڈکشن ایپلیکیشنز ڈیزائن کے ساتھ ship آپ کرتا ہے جو کہ پاس کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- **Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- **Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- **Model Context Protocol (MCP)** — backend integration کے لیے tools اور resources
- **Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- **Context windows** — لمبی دستاویزات کے ساتھ کام، multi-agent context passing
- **CI/CD pipelines** — automated code review، test generation
- **Escalation and reliability** — error handling، human-in-the-loop

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (4 درست 1 میں سے)
اسکورنگ	100–1000 scale، passing score **720**
Guessing penalty	نہیں (ہر سوال کا جواب دیں!)
Scenarios	8 4 میں سے (randomly selected)

امتحانی مواد: 5 ڈومینز

ڈومین	وزن
1. Agent architecture and orchestration	**27%**
2. Tool design and MCP integration	**18%**
3. Claude Code configuration and workflows	**20%**
4. Prompt engineering and structured output	**20%**
5. Context management and reliability	**15%**

امتحانی منظرنامہ

Scenario 1: Customer Support Agent

آپ ایک agent بنا رہے ہیں جو Claude Agent SDK استعمال کرتا ہوا returns، billing disputes، اور account issues کو سنبھالتا ہے۔ agent MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`) استعمال کرتا ہے۔ ہدف 80%+ first-contact resolution مناسب escalation کے ساتھ۔

Scenario 2: Claude Code کے ساتھ Code Generation

آپ Claude Code کو development میں استعمال کرتے ہیں تیز کرنے کے لیے: code generation، refactoring، debugging، documentation۔ آپ کو اس کے ساتھ CLAUDE.md configuration اور custom slash commands integrate کرنے کے لیے سمجھنا ہے کہ planning mode کب استعمال کرنا ہے، اور یہ کہ کب کرنا ہے۔

Scenario 3: Multi-Agent Research System

ایک coordinator agent specialized subagents کو tasks سونپتا ہے: web research، document analysis، synthesis، اور report generation۔ سسٹم کو citations کے ساتھ مکمل reports تیار کرنے چاہئیں۔

Scenario 4: Developer Productivity Tools

یہ agent engineers کو unfamiliar codebases explore کرنے، boilerplate code اور routine tasks بنانے، اور MCP servers اور Built-in tools (Read، Write، Bash، Grep، Glob) استعمال کرنے میں مدد دیتا ہے automate کیے جاتے ہیں۔

Scenario 5: Claude Code for Continuous Integration

Claude Code کو CI/CD pipeline میں ضم کریں تاکہ automated code reviews، test generation، اور pull request feedback حاصل ہو۔ Prompts کے ایسے ڈیزائن ہونے چاہئیں کہ false positives کم سے کم ہوں۔

Scenario 6: Structured Data Extraction

□□ معلومات نکالتا □ سے دستاویزات □ سسٹم غیر ساخت□، output کو JSON schemas □ ذریعہ □ کے validate ،□□ کرتا اور high accuracy □ اسے □□□ برقرار رکھتا edge cases □□ یہ□ چاہ□ کو درست طور پر سنبھالنا

Scenario 7: Conversational AI Architecture Patterns

آپ multi-turn conversational systems میں جن □یں □ ڈیزائن کرتے context window management، متعدد turns میں instructions □نا برقرار ر□ کا، memory strategies، محفوظ execution □ لیے □ کے tool design، اور مبہ□م یا متضاد user inputs □□□ کو سنبھالنا شامل

Scenario 8: Agentic AI Tools *(مواد موجود نہیں — ہماری مدد کریں!)*

اس scenario □ والے candidates کی اطلاع امتحان دینے □ ابھی تک اس □ لیکن یہ □□ دی □ نہ guide میں شامل نہ□یں کریں share میں GitHub Issues کے □ سوالات دیکھ□ □یں، تو ان□یں □□□ scenario اگر آپ نہ □ اصل امتحان میں اس □م مکمل □ تاکہ coverage کر تیاری کی امتحان جو گا □ مدد کر□ شخص کی اس ر□ تعاون کا آپ □ شامل کر سکیں ر□ا □□□

سرکاری دستاویزات

وسیلہ□	URL
Claude API — Messages	https://platform.claude.com/docs/en/api/messages
Claude API — Tool Use	https://platform.claude.com/docs/en/build-with-claude/tool-use
Claude API — Message Batches	https://platform.claude.com/docs/en/build-with-claude/message-batches
Claude Agent SDK — Overview	https://platform.claude.com/docs/en/agent-sdk/overview
Claude Agent SDK — Hooks	https://platform.claude.com/docs/en/agent-sdk/hooks
Claude Agent SDK — Subagents	https://platform.claude.com/docs/en/agent-sdk/subagents
Claude Agent SDK — Sessions	https://platform.claude.com/docs/en/agent-sdk/sessions
Model Context Protocol (MCP)	https://modelcontextprotocol.io/
MCP — Tools	https://modelcontextprotocol.io/docs/concepts/tools
MCP — Resources	https://modelcontextprotocol.io/docs/concepts/resources
MCP — Servers	https://modelcontextprotocol.io/docs/concepts/servers
Claude Code — Documentation	https://code.claude.com/docs/en/overview
Claude Code — CLAUDE.md and Memory	https://code.claude.com/docs/en/memory
Claude Code — Skills (incl. slash commands)	https://code.claude.com/docs/en/skills

Claude Code — Hooks	https://code.claude.com/docs/en/hooks
Claude Code — Sub-agents	https://code.claude.com/docs/en/sub-agents
Claude Code — MCP Integration	https://code.claude.com/docs/en/mcp
Claude Code — GitHub Actions CI/CD	https://code.claude.com/docs/en/github-actions
Claude Code — GitLab CI/CD	https://code.claude.com/docs/en/gitlab-ci-cd
Claude Code — Headless (non-interactive mode)	https://code.claude.com/docs/en/headless
Prompt Engineering Guide	https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
Extended Thinking	https://platform.claude.com/docs/en/build-with-claude/extended-thinking
Anthropic Cookbook (code examples)	https://github.com/anthropics/anthropic-cookbook

---

□صہ I: نظریاتی بنیادیں

یہ حصہ وہ تمام نظریہ احاطہ کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے مواد کو ٹیکنالوجیز اور concepts امتحانی کے مطابق ترتیب دیا گیا ہے، نہ کہ domains کے مطابق — اس سے آپ کو موضوع کی زیادہ گہری سمجھ پیدا کر سکتے ہیں

---

1 باب: Claude API — Model Interaction کی بنیادیں

> دستاویزات: Messages API | Prompt Engineering

1.1 API Request Structure

Claude API ایک request–response model ہے جو Claude Messages API کی پیروی کرتا ہے۔ ہر request میں یہ شامل ہوتا ہے:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

اہم فیلڈز:

- `model` — model selection (`claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5`)
- `max_tokens` — response میں زیادہ سے زیادہ tokens کی تعداد
- `system` — system prompt (model کے رویے کی تعریف کرتا ہے)
- `messages` — conversation history (**تسلسل برقرار رکھنے کے لیے آپ کو مکمل history بھیجنی ہوتی** ہے)
- `tools` — دستیاب tools کی definitions
- `tool_choice` — tool selection strategy

1.2 Message Roles

`messages` array دو conversational roles اور ایک instructional role استعمال کرتا ہے:

- `user` — user messages، بشمول tool results (ایک یا `user`-role message اندر کے `tool_result` content block کی صورت میں بھیجے جاتے ہیں، الگ `tool` role میں نہیں کے طور پر)
- `assistant` — model responses (history میں بھیجتے وقت شامل کیے جاتے ہیں)، بشمول tool use requests (`tool_use` content blocks)
- `system` — top-level `system` field کے ذریعے سیٹ کیا جا سکتا ہے (پہلے turn سے پہلے یا لاگو ہوتا ہے) اس نقطے سے آگے شامل کیا جا سکتا ہے (`messages` میں `{"role": "system", ...}` کے طور پر inline کے طور پر، مخصوص لاگو ہوتا ہے placement rules دیکھیں — نیچے کے تابع)

Tool results کو `"tool"` role والے message ایک یا `user`-role message کے طور پر نہیں بھیجا جاتا ہے کے طور پر بھیجے جاتے ہیں جس کے content میں ایک `tool_result` content block ہوتا ہے شامل:

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01...",
      "content": "..."
    }
  ]
}
```

`system` راست برا□□ `messages` array میں بھی ایک role صرف □ نہ، □□ ہو سکتا □و ظا□ر طور پر top-level کا□ `system` parameter اس کا مقصد □□ ذریعہ top-level کا□ field `system` cached prefix کو invalidate بغیر کیے□ placement rules میں□: conversation درمیان کا□ instructions مخصوص اس کا□ شامل کرنا □□□

- ایک □ یہ `user` turn (جس میں بشمول و□ `tool_result` blocks یا server tool use والا□ پر ختم □ونے□ turn `assistant` کا□ فوراً بعد آنا چا□یے□□
- کسی □ یہ `assistant` turn یا □ آنا چا□یے□ پہلا□ سے array کا آخری element □□ونا چا□یے□□
- کسی □ یہ `tool_use` block کا□ اس اور `tool_result` پر 400 ایسا کرنے□ — آ سکتا □یں درمیان کا□ error ملتا □□□
- پہلا□ والوں پر اور بعد (کا□ درمیان شامل کیے□ گئے□ conversation بشمول) messages `system` بعد میں آنے□ والا□ کا□ لیے□ turns کا□ top-level `system` field پر فوقیت رکھتے□ □یں□

انتہائی اہم: □ر API request **مکمل** میں آپ کو **conversation history** بھیجنی چا□یے□□ Model requests کا□ درمیان state □ر — محفوظ نہ□یں رکھتا call □□□ آزاد □وتا

1.3 Response میں `stop_reason` Field

Claude API response میں `stop_reason` کا□ □□ بتاتا □□ جو model نے□ generation شامل □وتا □□، کیوں روکی:

Value	Description	Action
`"end_turn"`	Model نے□ response مکمل کر لیا	نتیج□ user کو دکھائیں
`"tool_use"`	Model tool call □□ کرنا چا□تا	Tool execute کریں اور result واپس دیں
`"max_tokens"`	Token limit ختم □و گئی	Response truncated □□ ممکن ;□□ limit بڑھانی پڑ□
`"stop_sequence"`	Stop sequence مل گئی	اپنی application logic کا□ مطابق handle کریں

Agentic systems میں `"tool_use"` اور `"end_turn"` سب سے ا□م □یں — یہی agent loop کو کنٹرول کرتے□ □یں□

1.4 System Prompt

System prompt ایک خاص instruction جو □□ context اور behavioral rules □ یہ □□□ متعین کرتا□:

- `messages` array الگ □ یہ؛ □وتا نہ□یں حصہ□ کا `system` field □□ میں دیا جاتا

- user messages □□ پر فوقیت رکھتا
- ایک بار load اور پوری □وتا □□ conversation □□ میں لاگو ر□تا
- role، constraints، اور output format □□ کی تعریف ک□ لی□ استعمال □وتا □□

امتحان ک□ لی□ ا□م: system prompt کی wording غیر ارادی tool associations طور پر مثال ک□ پیدا کر سکتی □□□ "□میشہ customer □دایت جیسی تصدیق کریں" کو model کو `get_customer` کو ضرورت س□ زیاد□ استعمال کرن□ پر مجبور کر سکتی □□، حتیٰ ک□ جب ی□ ضروری ن□ □و□

1.5 Context Window

Context window text (tokens) □جس □□ کی کل مقدار model ایک وقت میں process اس میں □□□ کر سکتا □□ شامل:

- System prompt
- مکمل message history
- Tool definitions
- Tool results

ا□م context-window مسائل:

1. **Lost-in-the-middle effect:** models □لمبے input □ اعتماد س□ کم آغاز اور اختتام پر موجود معلومات کو زیاد□ کر سکت□ □یں□ حل: ا□م معلومات شروع یا آخر ک□ قریب miss کرت□ □یں، مگر درمیان کی تفصیلات process رکھیں□
2. **Tool results** ک□ **جمع □ونا:** tool call context میں output اگر □□□ شامل کرتا tool 40+ fields واپس کر□ ا□م □وں، تو مگر صرف 5 context □□□ کا بڑا حص□ ضائع □و جاتا
3. **Progressive summarization:** history compress کرت□ وقت numeric values، percentages، اور dates اکثر گم □و کر غیر واضح □و جات□ □یں ("تقریباً"، "کچھ"، "چند")□

---

2 باب: Tools اور `tool_use`

> دستاویزات: Tool Use

2.1 `tool_use` □□ کیا

`tool_use` ایک mechanism جو □□ Claude کو external functions call □□□ کی اجازت دیتا □□ برا□ Model کرن□ result کرتا □□ اور execute اس□ code بناتا □□؛ آپ کا structured tool call request نہ□یں چلاتا — وہ□ code راست واپس کرتا □□□

2.2 Tool Definition

ہر□ tool ایک JSON schema □ذریع□ ک□ define □□ کیا جاتا □□:

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile,
including name, email, order history, and account status. Use this tool BEFORE
lookup_order to verify the customer's identity. Accepts an email (format:
user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

Tool description لکھنے کا انتہائی اہم پہلو:

1. **Description tool selection کا بنیادی mechanism** ہے۔ LLM tools کو ان کی descriptions کی بنیاد پر منتخب کرتا ہے۔ Minimal descriptions ("Customer information retrieve کرتا ہے") اُن مواقع پر غلطیوں کا باعث بنتی ہیں جب tools کا ایک دوسرے سے overlap ہوں۔
2. **Description میں شامل کریں:**
   - Tool کیا کرتا ہے اور کیا واپس کرتا ہے
   - Input formats اور example values
   - Edge cases اور constraints
   - یہ tool similar alternatives کے مقابلے میں کب استعمال ہو
3. **اگر آپ کے tools ایک جیسے یا overlapping descriptions** رکھیں گے تو model تقریباً ایک جیسی descriptions میں خلط ملط کر دے گا۔ `analyze_document` اور `analyze_content` کی
4. **Built-in tools کے مقابلے MCP tools:** agents similar functionality کے لیے built-in tools (Read، Grep) کو MCP tools پر ترجیح دے سکتے ہیں۔ اس سے بچنے کے لیے MCP tool descriptions کو مضبوط بنائیں — concrete advantages، unique data، یا وہ context جو built-in tools نہیں کر سکتے فراہم نہیں نمایاں کریں۔

2.3 `tool_choice` Parameter

`tool_choice` کنٹرول کرتا ہے کہ model tools کو کیسے منتخب کرتا ہے:

Value	Behavior	کب استعمال کریں
`{ "type": "auto" }`	Model کو خود طے کرنا ہے کہ tool call کرنا ہے یا text میں جواب دینا ہے	default زیادہ تر حالات میں
`{ "type": "any" }`	Model کو لازماً کوئی tool call کرنا ہوگی	جب آپ کو guaranteed structured output چاہیے
`{ "type": "tool", "name": "extract_metadata" }`	Model کو لازماً ایک مخصوص tool call کرنی ہوگی	جب آپ کو forced first step / execution order چاہیے

□م منظرنام□ا:

- `tool_choice: "any"` + multiple extraction tools → model □ترین□ب tool منتخب کرتا □□، مگر پھر بھی structured output □□ ملتا
- Forced selection → جب آپ کو کسی خاص □لی پ□ی action □یا□ی کی ضمانت چاہ□ (مثلاً enrichment □ا□ پ□ سے پ□ا□ `extract_metadata`)

2.4 Structured Output □یا□ کے JSON Schemas

Claude □سے structured output **قابلِ سب سے□** حاصل کرنے□ کا `tool_use` استعمال کرنا JSON schemas کے□ ساتھ **اعتماد** طریقے□ □□□ ہے□:

- Syntax درست □سے اعتبار کے□ JSON □□ کی ضمانت دیتا (braces □وتے□ ہیں□ غائب، trailing commas ہیں□ نہ□وتے□)
- □مطلوبہ structure □□ نافذ کرتا (required fields ہیں□ □وتے□ موجود)
- Semantic correctness کی ضمانت نہ□یں دیتا (values ہیں□ □و سکتی□ ہیں پھر بھی غلط)

Schema design — بنیادی اصول:

```
{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
      "enum": ["bug", "feature", "docs", "unclear", "other"]
    },
    "category_detail": {
      "type": ["string", "null"],
      "description": "Details if category = 'other' or 'unclear'"
    },
    "severity": {
      "type": "string",
      "enum": ["critical", "high", "medium", "low"]
    },
    "confidence": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "optional_field": {
      "type": ["string", "null"],
      "description": "Null if the information was not found in the source"
    }
  },
  "required": ["category", "severity"]
}
```

Schema design rules:

1. **Required □بمقابل optional:** صرف اُن fields کو required □و□ بنائیں جن کی معلومات □میشہ□ دستیاب □و□ گھڑنے□ پر مجبور کر سکتی□ ہیں□ غائب values □ونے□ پر data کو Required fields model

2. **Nullable fields:** ایسی معلومات کے لیے `"type": ["string", "null"]` استعمال کریں جو ممکنہ طور پر موجود نہ ہوں Model `null` خود ساختہ بجائے اس کے کہ وہ واپس کر سکتا ہے value بنائے
3. **Enums with `"other"`:** اپنی predefined categories کے لیے سب باہر کی معلومات ضائع ہونے سے بچانے کے لیے ایک `"other"` + detail string شامل کریں
4. **Enum `"unclear"`:** جب اُن صورتوں کے لیے model category ایماندار — ہو نہ ہو کہ بار میں پُراعتماد نہ `"unclear"` غلط category سے بہتر ہے

2.5 Syntax بمقابلہ Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON، غلط field type	JSON schema کے ساتھ `tool_use` (ختم کر دیتا ہے)
Semantic	Totals match نہیں کرتے، value غلط field میں، hallucination	Validation checks، feedback کے ساتھ retry، self-correction

---

3 باب: Claude Agent SDK — Agentic Systems بنانا

> دستاویزات: Agent SDK | Hooks | Subagents | Sessions

3.1 Agentic Loop کیا ہے

Agentic loop خودکار task execution کا بنیادی pattern ہے Model — وہ صرف جواب نہیں دیتا actions کی ایک sequence انجام دیتا ہے:

```
1. Claude کو tools کے ساتھ request بھیجیں
2. Response وصول کریں
3. stop_reason چیک کریں:
   - "tool_use" -> tool execute کریں، result history میں append کریں، step 1 پر واپس جائیں
   - "end_turn" -> task مکمل، result user کو دکھائیں
4. مکمل ہونے تک repeat کریں
```

یہ model-driven approach ہے: Claude اپنے context اور پچھلے tool results کی بنیاد پر اگلا tool خود منتخب کرتا ہے یہ hard-coded decision trees کی جگہ لیتا ہے مختلف سے action sequence سے پہلے fixed ہوتی ہے

Anti-patterns (ان سے بچیں):

- Completion detect کرنے کے لیے assistant text parse کرنا (“Task completed”)
- Primary stop condition کے طور پر arbitrary iteration limit استعمال کرنا (مثلاً `max_iterations=5`)
- سمجھنا completion signal دیا ہے یا نہیں، اسے یہ چیک کرنا کہ assistant نے textual content

درست طریقہ: completion کا واحد قابلِ اعتماد signal `stop_reason == "end_turn"` ہے

3.2 `AgentDefinition` Configuration

`AgentDefinition` Claude Agent SDK میں agent configuration object ہے:

```
agent = AgentDefinition(
    name="customer_support",
    description="Handles customer requests for returns and order issues",
    system_prompt="You are a customer support agent...",
    allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
    # Coordinator میں کیا ہوگا:
    # allowed_tools=["Task", "get_customer", ...]
)
```

اہم parameters:

- `name` / `description` — agent کی شناخت اور وضاحت
- `system_prompt` — ہدایات والا system prompt
- `allowed_tools` — اجازت یافتہ tools کی فہرست (least privilege اصول)

3.3 Hub-and-Spoke: Coordinator اور Subagents

Multi-agent architecture عموماً hub-and-spoke topology میں بنائی جاتی ہے:

```
        Coordinator
       /     |      \
  Subagent1  Subagent2  Subagent3
   (search)   (analysis)   (synthesis)
```

Coordinator کی ذمہ داریاں:

- Task کو subtasks میں تقسیم کرنا
- کون سے subagents درکار ہیں (dynamic selection) یہ طے کرنا کہ کون سا
- subagents کو سونپنا کام
- نتائج جمع اور validate کرنا
- Errors اور retries handle کرنا
- نتائج user تک پہنچانا

اہم اصول: subagents کا context الگ ہوتا ہے

- Subagents خود بخود coordinator کی conversation history inherit نہیں کرتے
- تمام ضروری context subagent prompt میں **واضح طور پر** دینا ہوتا ہے
- Subagents calls درمیان کے memory share نہیں کرتے
- تمام communication coordinator کے ذریعے گزرتی ہے (observability اور error control کے لیے)

3.4 Subagents لیے کے بنانے `Task` Tool

Subagents `Task` tool کے ذریعے کے spawn میں جاتے ہیں کیے:

```
# Coordinator کے allowedTools میں "Task" ہونا چاہیے
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

Explicit context passing لازمی ہے:

```
# Bad: subagent پاس کے context نہیں ہے
Task: "Analyze the document"

# Good: مکمل context prompt میں
Task: "Analyze the following document.
Document: [full document text]
Prior search results: [web search results]
Output format requirements: [schema]"
```

Parallel spawning: coordinator ایک response میں متعدد `Task` s call کر سکتا ہے — subagents parallel میں چلتے ہیں:

```
# Coordinator ایک کے response میں یہ شامل ہے:
Task 1: "Search for articles about X"
Task 2: "Analyze document Y"
Task 3: "Search for articles about Z"
# تینوں ایک ساتھ چلتے ہیں
```

3.5 Agent SDK میں Hooks

Hooks agent lifecycle مخصوص کے points پر interception اور transformation کی اجازت دیتے ہیں

PostToolUse tool result کو model سے پہلے پہنچانے تک intercept کرتا ہے:

```
# مثال: مختلف MCP tools کے تاریخ سے formats normalize کریں
@hook("PostToolUse")
def normalize_dates(tool_result):
    # Unix timestamp -> ISO 8601
    # "Mar 5, 2025" -> "2025-03-05"
    return normalized_result
```

Outgoing-call interception hook policy کی خلاف ورزی کرنے والے actions کو block کرتا ہے:

```
# مثال: $500 سے اوپر refunds block کریں
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

Hooks اور prompt instructions میں فرق

Attribute	Hooks	Prompt instructions
Guarantee	**Deterministic** (100%)	**Probabilistic** (>90%, 100% نہیں)
کب استعمال کریں	Critical business rules، financial operations، compliance	General preferences، recommendations، formatting
مثال	Refunds > $500 block کریں	“Try to solve before escalating”

قاعدہ: جب failure ہوں کہ مالی، قانونی، یا حفاظتی نتائج ہیں — prompts نہیں، hooks استعمال کریں۔

باب 4: Model Context Protocol (MCP)

> دستاویزات: MCP | Tools | Resources | Servers

4.1 MCP کیا ہے

Model Context Protocol (MCP) ایک open protocol جو AI applications کو external systems سے جوڑتا ہے MCP تین بنیادی resource types define کرتا ہے:

1. **Tools** — functions جنہیں agent actions کے لیے call کر سکتا ہے (CRUD operations، API calls، command execution)
2. **Resources** — context کے لیے data جسے agent پڑھ سکتا ہے (documentation، database schemas، content catalogs)
3. **Prompts** — عام tasks کے لیے predefined prompt templates

4.2 MCP Servers

MCP server ایک process ہے جو MCP protocol implement کرتا ہے اور tools/resources فراہم کرتا ہے جب آپ کا MCP server سے connect ہوتا ہے:

- تمام tools خود بخود discover ہو جاتے ہیں
- تمام connected servers کے tools ایک ساتھ دستیاب ہوتے ہیں
- Tool descriptions کیسے استعمال کرنی ہیں model کی راہنمائی کرتی ہیں کہ

4.3 MCP Servers کو Configure کرنا

Project configuration (`.mcp.json`) — team usage □لیے □کہ:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    },
    "jira": {
      "command": "npx",
      "args": ["-y", "mcp-server-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

اہم نکات□:

- `.mcp.json` project root اور □□ ہوتا □ محفوظ میں version control □□ ہوتا □ میں
- Environment variables (`${GITHUB_TOKEN}`) secrets میں □ہوتے □ استعمال □لیے □کہ — tokens خود commit نہ□یں کیے جاتے□
- □ project تمام □کہ contributors □□ ہوتا □ دستیاب □لیے □کہ

User configuration (`~/.claude.json`) — personal/experimental servers □لیے □کہ:

- User □کہ home directory □□ ہوتا □ محفوظ میں
- Version control □ذریعے □کہ share ہوتا □ نہ□یں
- Personal experiments اور testing موزوں □لیے □کہ

Servers کا انتخاب:

- Standard integrations (Jira، GitHub، Slack) موجود □ سے □لا□ پ□لے □لیے □کہ community MCP servers کو ترجیح دیں
- اپنی custom servers صرف unique، team-specific workflows بنائیں □لیے □کہ

4.4 MCP میں `isError` Flag

جب MCP tool میں error تو □□ آتا response میں `isError: true` □ یہ □□□ ہوتا □ استعمال agent □کہ □□ کو بتاتا □□ call fail □□□ ہوئی□

Structured error (اچھا):

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable. Timeout while calling the
orders API.",
    "attempted_query": "order_id=12345",
    "partial_results": null
  }
}
```

Generic error (anti-pattern):

```
{
  "isError": true,
  "content": "Operation failed"
}
```

Generic error agent کیا — کوئی معلومات نہیں دیتا کہ لیے فیصلہ سازی کرے، retry کریں، query بدلیں، یا escalate کریں؟

4.5 MCP Resources

Resources ایسا data جو agent context کے لیے حاصل کرنے actions کے بغیر طلب کر سکتا ہے:

- Content catalogs (مثلاً تمام project tasks کی فہرست، hierarchical navigation)
- Database schemas (data structure کے لیے سمجھنے)
- Documentation (API references، internal guides)
- Issue/task summaries

Resource کا فائدہ: agent کے لیے سمجھنے کو یہ exploratory tool calls کی ضرورت نہیں پڑتی کہ data موجود کیا Resource فوری “map” کرتا فراہم ہے

باب 5: Claude Code — Configuration اور Workflows

> دستاویزات: Claude Code | Memory / CLAUDE.md | Skills | MCP | Hooks | Sub-agents | GitHub Actions | Headless

5.1 CLAUDE.md Hierarchy

CLAUDE.md □و instruction file(s) جو □یں Claude Code اس کی تین سطحی hierarchy استعمال کرتا □□□ □□:

```
1. User-level: ~/.claude/CLAUDE.md
   - صرف اسی user پر لاگو
   - VCS کہ ذریعہ share نہیں ہوتا
   - ذاتی preferences اور working style

2. Project-level: .claude/CLAUDE.md یا root CLAUDE.md
   - تمام project contributors پر لاگو
   - VCS کہ ذریعہ manage □□ ہوتا
   - Coding standards، testing standards، architectural decisions

3. Directory-level: subdirectories میں CLAUDE.md
   - جب اسی directory کی files □□ ہوتا تو لاگو ہو کہ ساتھ کام ہو
   - codebase کہ مخصوص کہ اُس حصہ conventions
```

□ئک ایک نئ **عام غلطی:** team member کو project instructions □و □یں ملتیں کیونک□ .claude/CLAUDE.md (project-level) □ کہ بجائ□ ~/.claude/CLAUDE.md (user-level) □ میں رکھی گئی تھیں

5.2 @path Syntax (File Imports)

CLAUDE.md بیرونی files کو @path □ ذریع□ کہ refer □ جس س□، □□ کر سکتا configuration modular □و جاتی □□:

```
# Project CLAUDE.md

Coding standards @./standards/coding-style.md میں بیان کیہ گئہ ہیں
Test requirements @./standards/testing-requirements.md میں ہیں
Project overview @README.md اور میں dependencies @package.json میں ہیں
```

@path قواعد □ک:

- @ فوراً بعد □ک file path □آئ (کوئی space □یں)
- Relative اور absolute دونوں paths supported □یں
- Relative paths اُس file مطابق □ک resolve میں جس □یں □وت□ import □□
- Maximum import nesting depth 5 □□

□س اس duplication □ر اور □وتی □□ کم package میں صرف متعلق□ standards □یں □وت□ شامل

5.3 .claude/rules/ Directory

.claude/rules/ monolithic CLAUDE.md جو ،□□ متبادل کا rules کو topic □ حساب س□ □ک organize □ کرن□ لی□ استعمال □وتا □□:

```
.claude/rules/
  testing.md          -- testing conventions
  api-conventions.md  -- API conventions
  deployment.md       -- deployment rules
  react-patterns.md   -- React patterns
```

YAML frontmatter ساتھ کے `paths` conditional loading:

```
---
paths: ["src/api/**/*"]
---

API files کے لیے explicit error handling استعمال کریں
ہر endpoint standard response wrapper واپس کرے
```

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---

Tests میں describe/it blocks استعمال ہونے چاہئیں
Data factories استعمال کریں، hardcoding نہیں
Database mock نہ کریں — test database استعمال کریں
```

یہ کیسے کام کرتا ہے:

- Rule صرف اس وقت load ہوتا ہے جب Claude Code ایسا file edit جو `paths` pattern سے match کرتا ہو
- اس سے context اور tokens غیر متعلقہ rules load ہوتے ہیں — بچت
- Glob patterns آپ کو file type کے لحاظ سے conventions apply کرنے دیتے ہیں، چاہے files codebase میں کہیں بھی ہوں (tests مفید ہے)

`.claude/rules/` بمقابلہ directory-level CLAUDE.md والا `paths` — کب استعمال کریں:

- `.claude/rules/` with `paths` — جب conventions کئی directories میں بکھری files (tests، migrations) پر لاگو ہوں
- Directory-level CLAUDE.md — جب conventions کسی خاص directory اور اس سے وابستہ ہوں اور کہیں ضروری نہ ہوں

5.4 Custom Slash Commands اور Skills

> **نوٹ:** موجود Claude Code version میں custom commands (`.claude/commands/`) کو skills (`.claude/skills/`) کے ساتھ unify کر دیا گیا ہے۔ دونوں `/name` commands بناتے ہیں اور اب بھی supported ہے۔ format کا ذکر کرتی ہے — وہ `.claude/commands/` امتحانی گائیڈ ہے۔

Slash commands reusable prompt templates ہیں جو `/name` کے ذریعے invoke ہوتے ہیں:

`.claude/commands/` format (legacy، supported):

```
.claude/commands/
  review.md        -- /review -- standard code review
  test-gen.md      -- /test-gen -- test generation
```

`.claude/skills/` format (current):

```
.claude/skills/
  review/SKILL.md  -- /review -- frontmatter configuration کہ ساتھ
  test-gen/SKILL.md
```

Project commands (`.claude/commands/` یا `.claude/skills/`):

- VCS اور repo clone میں محفوظ ہوتی ہیں دستیاب ہوتی ہیں کرنے والی سب کی لیے
- ٹیم میں consistent workflows یقینی بناتی ہیں

User commands (`~/.claude/commands/` یا `~/.claude/skills/`):

- ذاتی commands جو VCS کی ذریعے کی share ہوتی نہیں
- انفرادی workflows کی لیے

5.5 Skills — `.claude/skills/`

Skills advanced commands جو ہیں SKILL.md frontmatter کی ذریعے کی configure ہیں ہوتی:

```
---
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
argument-hint: "Path to the directory to analyze"
---

دی گئی directory میں code structure کا تجزیہ کریں۔
Dependencies اور architectural patterns پر ایک report تیار کریں۔
```

Frontmatter parameters:

Parameter	Description
`context: fork`	Skill کو isolated subagent میں چلاتا ہے۔ Verbose output main session کو pollute نہیں کرتا
`allowed-tools`	دستیاب ہوں گے tools جو محدود کرتا ہے کہ کون سے (security — مثلاً skill files delete نہیں کر سکتی اگر اجازت نہ ہو)
`argument-hint`	جب skill parameters بغیر کے invoke ہو تو argument کا اشارہ مانگتا ہے

Skill بمقابلہ CLAUDE.md کب استعمال کریں:

- **Skill** — مخصوص task کی لیے on-demand invocation (review، analysis، generation)
- **CLAUDE.md** — ہمیشہ لوڈ ہونے والی general standards اور conventions

Personal skills (`~/.claude/skills/`):

- اپنی personal variants □کو بنائیں تاکہ مختلف ناموں سے teammates □وں متاثر نہ □وں

5.6 Planning Mode □بمقابلہ Direct Execution

Planning mode:

- Model صرف investigate اور plan □□ کرتا changes کرتا نہ□یں؛
- Codebase explore □لیے □کو □کرنے Read، Grep، Glob □□ استعمال کرتا
- ایک implementation plan □جسے □□ تیار کرتا user approve □□ کرتا
- بغیر side effects □کے safe exploration

Planning mode کب استعمال کریں:

- □بڑے changes (درجنوں files)
- Multiple plausible approaches (microservices: boundaries □کیسے define □ہوں؟)
- Architectural decisions (کون سا framework؟ کیا structure؟)
- Unfamiliar codebase (change □□ سمجھنا ضروری □□□ پہلے □کرنے)
- Library migrations 45 جو+ files کو متاثر کریں

Direct execution کب استعمال کریں:

- Single-file fixes واضح stack trace ساتھ □کے
- ایک validation check شامل کرنا
- اچھی طرح سمجھے □گئے، غیر مبہ□م changes

Combined approach:

1. Investigation اور design □لیے □کے planning mode
2. User plan approve □کرے
3. Approved plan implement □لیے □کو کرنے direct execution

Explore subagent — codebase explore □لیے □کو کرنے specialized subagent:

- Verbose output کو main context □سے isolate □□ کرتا
- صرف summary □□ واپس کرتا
- Multi-phase tasks میں context-window exhaustion □□ روکتا

5.7 `/compact` Command

`/compact` context compress □لیے □کو کرنے built-in command □□:

- Pچھلی history کو summarize □کر کے context window □□ میں جگہ□ خالی کرتا
- Long investigation sessions جب □□ہوتا میں استعمال verbose tool output □سے context □بھر جائے
- Risk: exact numeric values، dates، اور specific details summarization میں □ہو سکتے □ہیں میں گم

5.8 `/memory` Command

`/memory` sessions درمیان کے memory manage کرنے کے لیے built-in command ہے:

- `CLAUDE.md` file editing کے لیے کھولتا ہے، تاکہ notes، preferences، اور context محفوظ کیے جا سکیں
- Information sessions اور پر خود بخود load ہوتی ہے کے startup پر برقرار رہتی ہے
- Project conventions، user preferences، frequently used commands، اور current work context محفوظ کرنے کے لیے مفید
- ہر session میں وہی instructions کا متبادل سمجھانے دوبارہ

5.9 Claude Code CLI for CI/CD

`-p` (یا `--print`) flag:

```
claude -p "Analyze this pull request for security issues"
```

- Non-interactive mode: prompt process کرتا ہے، stdout پر print کرتا ہے، اور exit ہو جاتا ہے
- User input کا انتظار نہیں کرتا
- CI/CD pipelines میں Claude چلانے کا واحد درست طریقہ

CI کے لیے structured output:

```
claude -p "Review this PR" --output-format json --json-schema
'{"type":"object",...}'
```

- `--output-format json` — output کو JSON میں دیتا ہے
- `--json-schema` — output کو schema مطابق کے validate کرتا ہے
- Result کو automatically inline PR comments پوسٹ کرنے کے لیے parse کیا جا سکتا ہے

Session context isolation: وہی Claude session جس نے code generate کیا ہو، اکثر review میں کم مؤثر ہوتا ہے (model اپنی reasoning context کو ساتھ رکھتا ہے challenge اور اپنے فیصلوں کو کرنے کا امکان کم ہوتی ہے)۔ Review کے لیے independent instance استعمال کریں

Duplicate comments سے بچاؤ: نئے commits کے بعد دوبارہ review کرتے وقت، پچھلے review results context کرنے کی ہدایت دیں کو صرف نئے یا unresolved issues report میں شامل کریں اور Claude

5.10 `fork_session` اور Session Management

`--resume <session-name>` named session کو resume کرتا ہے:

```
claude --resume investigation-auth-bug
```

- saved context کے ساتھ پچھلی conversation جاری رکھتا ہے

- Long investigations جو متعدد sessions پر پھیلی ہوں ان کے لیے مفید
- Risk: اگر پچھلی session بعد کے files تو بدل چکی ہوں tool results stale ہو سکتے ہیں

`fork_session` shared context سے independent branch بناتا ہے:

```
Codebase investigation
         |
    fork_session
    /           \
Approach A:      Approach B:
Redux            Context API
```

- دونوں forks branch point تک context inherit کرتے ہیں
- اس کے بعد وہ آزادانہ طور پر diverge کرتے ہیں
- Approaches compare یا strategies test مفید لیے کرنے کے

Resume نئی بجائے کے session کب شروع کریں:

- Tool results stale ہوں (files بدل چکی ہوں)
- اور ہو چکا کافی وقت گزر context degrade ہو گیا ہو
- پرانے tool data ساتھ کے resume :ہے یہ بہتر ہے کہ "ہم نے جو پایا اس کا مختصر خلاصہ کرنے کے بجائے یہ سے "... restart کیا جائے

---

6 باب: Prompt Engineering — Advanced Techniques

> دستاویزات: Prompt Engineering | Anthropic Cookbook

6.1 Few-shot Prompting

Few-shot prompting جس میں ہے طریقہ وہ expected behavior لیے کے دکھانے prompt 4–2 میں input/output examples شامل کیے جاتے ہیں

Few-shot text descriptions سے زیادہ مؤثر کیوں ہے:

- مبہم instruction زیادہ “جیسے precise ہے سمجھا جا سکتا سے کئی طریقوں کو” ہوں
- Example غیر مبہم طور پر expected format اور decision logic ہے دکھاتا
- Model pattern نئے کو cases پر generalize ہے کرتا (صرف examples ہیں نہ دہراتا)

Few-shot examples کی اقسام اور استعمال:

1. **Ambiguous scenarios لیے کے examples:**

```
Request: "My order is broken"
Action: Call get_customer -> lookup_order -> check status.
Rationale: “broken” damaged item کو آپ ؛؛ بھی ہو سکتا order details درکار ہیں۔

Request: "Get me a manager"
Action: فوراً escalate_to_human call کریں.
Rationale: Customer پر واضح طور پر human کی کوشش نہ کریں۔ مانگ رہا ؛؛؛ خود حل کرنہ
```

2. **Output formatting کے لیے examples:**

```
Finding example:
{
  "location": "src/auth/login.ts:42",
  "issue": "SQL injection in the username parameter",
  "severity": "critical",
  "suggested_fix": "Use a parameterized query"
}
```

3. **Acceptable بمقابلہ problematic code کے لیے کرنے الگ examples:**

```
// Acceptable (flag کریں نہ):
const items = data.filter(x => x.active);

// Problem (flag کریں):
const items = data.filter(x => x.active == true); // Use strict equality ===
```

4. **Different document formats سے extraction کے examples:**

```
Document with inline citations:
"As shown in the study (Smith, 2023), the rate is 42%."
-> {"value": "42%", "source": "Smith, 2023", "type": "inline_citation"}

Document with bibliography references:
"The rate is 42%. [1]"
-> {"value": "42%", "source": "reference_1", "type": "bibliography"}
```

5. **Informal measurements کے لیے examples:**

```
Text: "about two handfuls of rice"
-> {"amount": "~100g", "original_text": "two handfuls", "precision": "approximate"}

Text: "a pinch of salt"
-> {"amount": "~1g", "original_text": "a pinch", "precision": "approximate"}
```

Few-shot خاص طور پر informal اور non-standard measurement units جو ہیں متنوع بہت لیے کے instructions purely rule-based نکالنے میں مؤثر ہیں۔

Prompts میں format normalization rules: جب structured output کے لیے strict JSON schemas استعمال کریں، prompt میں normalization rules شامل کریں:

```
Normalization:
- Dates: ہمیشہ ISO 8601 (YYYY-MM-DD); "yesterday" -> absolute date compute کریں
- Currency: numeric amount + currency code; "five bucks" -> {"amount": 5, "currency":
"USD"}
- Percentages: decimal fraction; "half" -> 0.5
```

□اس سے semantic errors جیسی جگہوں □ہوتی □ہیں کم JSON syntactically □ہو مگر درست values inconsistent □ہوں□

6.2 Explicit Criteria □بمقابلہ Vague Instructions

Bad (vague):

```
Code comments کی accuracy چیک کریں۔
Conservative صرف — رہیں high-confidence findings report کریں۔
```

Good (explicit criteria):

```
Comment کو problematic صرف اُس صورت میں flag کریں جب:
1. Comment بیان کا actual code behavior سے CONTRADICT ہو کرتا
2. Comment کسی غیر موجود function یا variable کا حوالہ دے
3. TODO/FIXME comment کسی ایسے bug کی طرف اشارہ کرے جو code میں پہلے ہی fix ہو چکا ہو

Flag نہ کریں:
- ایسے comments جو صرف style کے لحاظ سے پرانے ہوں
- معمولی wording inaccuracies
- Missing comments (یہ الگ category ہے)
```

Severity criteria examples کے ساتھ □define کریں:

```
CRITICAL: User کے لیے runtime failure
  Example: Payment process کرتے وقت NullPointerException

HIGH: Security vulnerability
  Example: SQL injection، XSS، missing authorization checks

MEDIUM: Logic bug بغیر فوری impact کے
  Example: Wrong sorting، off-by-one error

LOW: Code quality
  Example: Duplication، چھوٹے data کے لیے suboptimal algorithm
```

6.3 Prompt Chaining

Prompt chaining ایک complex task کو focused steps کی sequence □□ میں توڑتا:

```
Step 1: auth.ts analyze کریں (صرف local issues)
       -> Output: auth.ts میں issues کی list

Step 2: database.ts analyze کریں (صرف local issues)
       -> Output: database.ts میں issues کی list

Step 3: Integration pass (cross-file dependencies)
       -> Output: module boundaries پر issues
```

یہ کیوں اہم ہے:

- **Attention dilution** جب—ایک ساتھ بہت سی files تو کچھ files دیکھتا ہے تو bugs miss کر سکتا ہے اور باقی پر shallow commentary دے سکتا ہے
- ہر file کے لیے consistent analysis quality یقینی بناتا ہے
- Cross-file interactions الگ کے لیے analysis کی اجازت دیتا ہے

Prompt chaining بمقابلہ dynamic decomposition کب استعمال کریں:

- **Prompt chaining** — predictable، repeatable tasks (code review، file migrations)
- **Dynamic decomposition** — open-ended investigations جہاں subtasks execution ہوتے کے دوران واضح ہیں

6.4 “Interview” Pattern

Solution implement کرنے سے پہلے Claude clarifying questions پوچھتا ہے:

```
Claude: "API کے لیے caching implement کرنے سے پہلے چند سوالات:
1. آپ TTL یا event-based میں سے کون سی cache invalidation strategy پسند کرتے ہیں؟
2. اگر cache unavailable ہو تو کیا stale data قابلِ قبول ہے؟
3. Caching per-user ہونی چاہیے یا global؟
4. Cache کے لیے expected data volume کیا ہے؟"
```

یہ کب مفید ہے:

- Unfamiliar domain (fintech، healthcare، legal systems)
- غیر واضح implications والے tasks (cache strategies، failure modes)
- متعدد viable approaches انتخاب بہترین جہاں context پر منحصر ہو

6.5 Validation اور Retry-with-Feedback

جب extracted data validation fail کرے:

```
Step 1: Document سے data extract کریں
Step 2: Validate کریں (Pydantic، JSON Schema، business rules)
Step 3: اگر error ہو — context کے ساتھ retry کریں:
  - Original document
  - Previous (incorrect) extraction
  - Specific error: "Field 'total' = 150, but sum(line_items) = 145. Re-check values."
```

Retry کب مؤثر ہوگا:

- Format errors (date غلط format میں)
- Structural errors (field □گ غلط)
- Arithmetic inconsistencies (model □بار□ check □□ سکتا کر)

Retry گا □کر □یں ن□ مدد کب:

- Information source document □ی ن□یں میں موجود
- Required context external □□ (data اور document کسی گیا کیا ن□یں فرا□م جو □□ میں)

Pydantic بطور validation tool: Pydantic Python library جو □□ schema-based data validation □لی □کہ استعمال □وتی □□□ امتحان کہ لیے ا□م نکات یہ □یں:

- **Structural validation:** types، requiredness، enum constraints JSON بعد □کہ code میں □جاتی □ چیک کیے میں
- **Semantic validation:** custom validators business logic enforce □یں □کرتی (sum of items equals total؛ start_date < end_date)
- **Validate–retry loops:** Pydantic validation failure پر error message بنا کر Claude کو error context □کہ ساتھ prompt کریں دوبار□
- **JSON Schema generation:** Pydantic models `tool_use` □لی □کہ JSON Schema generate □کر سکتی □یں، جو ایک single source of truth □□ فرا□م کرتا

6.6 Self-correction

Internal contradictions detect کا □کرنا pattern:

```
{
  "stated_total": "$150.00",
  "calculated_total": "$145.00",
  "conflict_detected": true,
  "line_items": [
    {"name": "Widget A", "price": 75.00},
    {"name": "Widget B", "price": 70.00}
  ]
}
```

Model stated value اور computed value تو □وں مختلف □ اگر و□ — □□ نکالتا دونوں `conflict_detected` آپ discrepancy handle □□□ دیتا □کرنا کو

---

7 باب: Message Batches API

> دستاویزات: Message Batches

7.1 Overview

Message Batches API آپ کو asynchronous processing □لی □کہ requests □کہ batches submit □□ دیتا □کرنا:

Attribute	Value
Savings	synchronous calls **50** میں مقابلہ کم **50%**
Processing window	**24** گھنٹے سے زیادہ **hours** (latency SLA guarantee نہیں)
Multi-turn tool calling	**Supported** نہیں (ایک request = ایک response)
Correlation	request اور response کو جوڑنے کے لیے `custom_id` field

7.2 Batch API بمقابلہ Synchronous API کب استعمال کریں

Task	API	Why
Pre-merge PR check	**Synchronous**	Developer 24 گھنٹے انتظار کر رہا ہے؛ hours قابلِ قبول نہیں
Overnight tech-debt report	**Batch**	Result 50% savings صبح تک چاہیے؛
Weekly security audit	**Batch**	50% savings فوری نہیں؛
Interactive code review	**Synchronous**	فوری response درکار ہے
10,000 documents process کرنا	**Batch**	Bulk processing؛ savings 50% ملے

7.3 `custom_id` کا استعمال

```
{
  "custom_id": "doc-invoice-2024-001",
  "params": {
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "Extract data from: ..."}]
  }
}
```

`custom_id` آپ کو یہ کرنے دیتا ہے:

- Result کو original document سے link کرنا
- Failure کی صورت میں صرف failed documents کو دوبارہ submit کرنا
- Successful documents کو دوبارہ process کرنے سے بچنا

7.4 Batch میں Failures کو Handle کرنا

1. 100 documents کا batch submit کریں
2. 95 succeed کریں؛ 5 fail ہوں (context limit exceeded)
3. Failures کو `custom_id` سے identify کریں
4. Strategy بدلیں (مثلاً long documents کو chunks میں تقسیم کریں)

5. 5 صرف اُن failed documents کو دوبارہ submit کریں

7.5 SLA Planning

اگر آپ کو 30 hours میں result اور چاہیے Batch API 24 کو hours میں تک لگ سکتے ہیں:

- Submission window: 30 - 24 = **6 hours**
- Batches کو deadline 24 کم از کم سے hours پہلے submit کرنا ہوگا
- Frequent submissions 4 لیے کے 1-hour windows میں تقسیم کریں

باب 8: Task Decomposition Strategies

8.1 Fixed Pipelines (Prompt Chaining)

ہر step سے پہلے defined ہوتا ہے:

```
Document -> Metadata extraction -> Data extraction -> Validation -> Enrichment -> Final output
```

کب استعمال کریں:

- Task structure predictable ہو (reviews کی ایک جیسی template follow کریں)
- تمام steps وں معلوم ہوں سے پہلے
- آپ کو stability اور reproducibility چاہیے

8.2 Dynamic Adaptive Decomposition

Subtasks intermediate results کی بنیاد پر generate ہوتے ہیں:

```
1. "Legacy codebase کے لیے tests add کریں"
2. -> پہلے structure map کریں (Glob, Grep)
3. -> 3 ملا: modules بغیر tests 2 ،کے partial coverage کے ساتھ
4. -> Prioritize: payments module کریں شروع سے (high risk)
5. -> کام کے دوران: external API پر dependency دریافت ہوئی
6. -> Adapt: tests لکھنے سے پہلے external API کے لیے mock add کریں
```

کب استعمال کریں:

- Open-ended investigative tasks
- شروع میں معلوم نہ ہو scope جب مکمل
- ہر جب step پچھلے step کے results پر منحصر ہو

8.3 Multi-pass Code Review

10+ files والی pull requests □لیے □کہ:

```
Pass 1 (per-file): auth.ts analyze کریں -> local issues کی list
Pass 1 (per-file): database.ts analyze کریں -> local issues کی list
Pass 1 (per-file): routes.ts analyze کریں -> local issues کی list
...
Pass 2 (integration): files درمیان کہ relationships analyze کریں
  -> Cross-file issues: inconsistent types, circular dependencies
```

14 files □ی پر ایک pass □□ خراب □کیوں:

- Attention dilution: کچھ files کی deep analysis، کچھ کی shallow
- Inconsistent comments: ایک file میں pattern flag، دوسری میں approve
- Missed bugs: cognitive overload واضح سے □□ کی وجہ errors □یں چھوٹ جاتے

باب 9: Escalation اور Human-in-the-Loop

9.1 کب Human کی طرف Escalate کریں

Escalation triggers (واضح rules):

Situation	Action
Customer □□کہ واضح طور پر “get me a manager”	فوراً escalate کریں؛ حل کرنے □ کی کوشش نہ کریں
Policy request کو cover □و نہ کرتی	Escalate کریں (مثلاً competitor price matching جب policy خاموش □و)
Agent □کہ سکے پیش رفت نہ کر	مناسب تعداد میں attempts بعد □کہ escalate کریں
Threshold اوپر سے financial operation	Escalate کریں (ترجیحاً hook □ذریعے □کہ enforce، prompt □ذریعے □کہ نہ□یں)
Customer وقت تلاش کرتے multiple matches	مزید identifiers مانگیں؛ اندازہ نہ لگائیں

جو reliable trigger □□ نہ□یں:

Unreliable method	Why it fails
Sentiment analysis	Customer کا mood case complexity سے correlate نہ□یں کرتا
Model self-rated confidence (1–10)	Model confidently غلط □و سکتا □□؛ calibration □□ کمزور
Automatic classifier	Overengineering□□ ممکن ؛ training data □و نہ موجود

9.2 Escalation Patterns

Immediate escalation:

```
Customer: "I want to speak to a manager"
Agent: [فوراً escalate_to_human call ▫▫ کرتا]
NOT: "I can help with your issue, let me..."
```

Resolve بعد □ کی کوشش کرنا□ escalation:

```
Customer: "My refrigerator broke two days after purchase"
Agent: [order ▫▫ چیک کرتا، warranty replacement offer ▫▫ کرتا]
اگر customer ہو ▫ مطمئن نہ ▫ -> escalate
```

Nuanced escalation (acknowledge → resolve → reiteration پر escalate):

```
Customer: "This is outrageous, I'm very unhappy with the quality!"
Agent: [frustration acknowledge] "میں آپ کی frustration ▫وں سمجھتا."
       [resolution offer] "میں replacement یا refund ▫وں پیش کر سکتا."
Customer: "No, I want to talk to someone!"
Agent: [customer ▫دوبارہ insist ▫کرہ -> immediate escalation]
```

□□پ :بنیادی اصول emotion acknowledge پھر کریں، concrete solution وقت اسی اور صرف دیں، escalate کریں جب customer human □□رائ□□ش د□ کی خوا□ dissatisfaction پر اظ□ار پ□□ کہ escalate کریں نہ□ (یہ manager □□ میں برابر نہ□ کہ□ مانگنا□)□

Policy gap □یہ □ کہ escalation:

```
Customer: "Competitor X has this item 30% cheaper—give me a discount"
Policy: صرف اپنی site پر price adjustments cover ▫▫ کرتی
Agent: [escalate — policy competitor price matching کو cover نہ▫یں کرتی]
```

9.3 Structured Handoff Protocols

Escalation پر agent کو structured summary human □یہ□ چا□ دینی کو:

```
{
  "customer_id": "CUST-12345",
  "customer_name": "Ivan Petrov",
  "issue_summary": "Refund request for a damaged item",
  "order_id": "ORD-67890",
  "root_cause": "Item arrived damaged; photos attached",
  "actions_taken": [
    "Verified customer via get_customer",
    "Confirmed order via lookup_order",
    "Offered a standard replacement — customer insists on a refund"
  ],
  "refund_amount": "$89.99",
  "recommended_action": "Approve a full refund",
  "escalation_reason": "Customer requested to speak with a manager"
}
```

Human operator کو full conversation transcript □یں صرف یہ □وتی — وہ summary □□□ دیکھتا تک رسائی نہ□یں □ونی چاہ□یے□□ self-contained اس لیے یہ □ مکمل اور

9.4 Confidence Calibration اور Human Oversight

Data extraction systems □یے □کہ:

1. **Field-level confidence scores:** model ہر extracted field □یے □کہ confidence score □□ نکالتا
2. **Calibration:** labeled validation sets □کہ استعمال کر کے thresholds tune کریں
3. **Routing:**
   - High confidence + stable accuracy -> automated processing
   - Low confidence یا ambiguous sources -> human review

Stratified random sampling:

- High-confidence extractions □ سے باقاعدگی بھی □یے □کہ sample audit کریں
- Aggregate 97% accuracy کسی مخصوص document type 40 □یے □کہ% errors □□ چھپا سکتی
- Accuracy کو صرف overall نہ□یں، document type اور field □ حساب سے □کہ analyze کریں

---

10 باب: Multi-agent Systems میں Error Handling

10.1 Error Categories

Category	Examples	Retryable	Agent action
Transient	Timeout، 503، network failure	Yes	Exponential backoff ساتھ □کہ retry
Validation	Invalid input format، missing required field	No (input fix کریں)	Request modify کریں اور retry کریں

Business	Policy violation، threshold exceeded	No	User کو سمجھائیں؛ alternative پیش کریں
Permission	Access denied	No	Escalate کریں

10.2 Error-handling Anti-patterns

Anti-pattern	Problem	Correct approach
Generic status “search unavailable”	Coordinator decide □یں کر سکتا کہ recover □□ کرنا □کیسے	Error type، query، partial results، alternatives واپس کریں
Silent suppression (empty result = success)	Coordinator □□ سمجھتا match □یں ملا، حالانکہ failure تھا □وا	“search failure” اور “no results” میں فرق کریں
ایک failure پر پورا workflow abort کرنا	تمام partial results □یں □و جاتے □یں ضائع	Partial results □ ساتھ جاری رکھیں؛ gaps annotate کریں
Subagent میں infinite retries	Latency اور resources کا ضیاع	Local recovery (1–2 retries)، پھر coordinator تک propagate

10.3 Structured Subagent Error

```
{
  "status": "partial_failure",
  "failure_type": "timeout",
  "attempted_query": "AI impact on music industry 2024",
  "partial_results": [
    {"title": "AI Music Generation Report", "url": "...", "relevance": 0.8}
  ],
  "alternative_approaches": [
    "Try a narrower query: 'AI music composition tools'",
    "Use an alternative data source"
  ],
  "coverage_impact": "Not covered: AI impact on music production"
}
```

□ی coordinator □□ ضروری معلومات دیتا □□ لیے □□ کہ فیصلے □□ کو:

- Modified query ساتھ □ retry کریں؟
- Partial results استعمال کریں؟
- کسی اور subagent کو delegate کریں؟
- اس section بغیر جاری رکھیں اور □ gap annotate کریں؟

10.4 Final Synthesis میں Coverage Annotations

```
## Report: AI Impact on Creative Industries

### Visual Art (FULL COVERAGE)
[research results]

### Music (PARTIAL COVERAGE — search agent timeout)
[partial results]
⚠️ Note: اس section کی coverage search agent timeout □□ محدود □ سے □ وجہ کی.

### Literature (FULL COVERAGE)
[research results]
```

---

11 باب: Production Systems میں Context Management

11.1 Facts کو Separate Block میں Extract کریں

Conversation history □ئ بجائ □ کہ □ پر انحصار کرنا (جو summarization دوران □ کہ degrade □□ جاتی □و)، key facts کو structured block میں extract کریں:

```
=== CASE FACTS (کیا جائ□ update آئ□ fact جب بھی نیا) ===
Customer ID: CUST-12345
Order ID: ORD-67890
Order Date: 2025-01-15
Order Amount: $89.99
Issue: Damaged item on delivery
Customer Request: Full refund
Status: Pending manager approval
===
```

□ی block ہ□ prompt □□چا، میں شامل کریں history □ی کتنی summarized □و□ □ نہ کیوں

11.2 Tool Results کو Trimming کرنا

اگر `lookup_order` 40+ fields مگر □□ واپس کرتا current task □یں □ درکار 5 صرف □ لی□ کہ:

```
# PostToolUse hook: صرف relevant fields رکھیں
@hook("PostToolUse", tool="lookup_order")
def trim_order_fields(result):
    return {
        "order_id": result["order_id"],
        "status": result["status"],
        "total": result["total"],
        "items": result["items"],
        "return_eligible": result["return_eligible"]
    }
```

□س اس context اور □□ بچتا noise □□□ کم □وتی

11.3 Position-aware Input

Lost-in-the-middle effect □وئ□ □ن ذ□کو میں رکھتہ□ critical information رکھیں:

```
[KEY FINDINGS — اوپر]
3 critical vulnerabilities ملیں...

[DETAILED RESULTS — درمیان]
=== File auth.ts ===
...
=== File database.ts ===
...

[ACTION ITEMS — آخر میں]
Priority: merge ہ□ پہ□ہ سہ auth.ts vulnerabilities fix کریں.
```

11.4 Scratchpad Files

Long investigations میں agent key findings scratchpad file □□ میں لکھ سکتا:

```
# investigation-scratchpad.md
## Key findings
- PaymentProcessor in src/payments/processor.ts inherits from BaseProcessor
- refund() تین places سہ call ہ□ ہ□وتی: OrderController، AdminPanel، CronJob
- External PaymentGateway API limit: 100 req/min
- Migration #47 نہ refund_reason (NOT NULL) add 01-12-2024 — کیا
```

جب context degrade □و جائ□ (یا نئی session میں)، agent discovery □بجائ□ کہ□ چلان□ دوبار□ scratchpad consult □□□ کر سکتا

11.5 Context □یا □ کے □بچانے کے لیے Subagents کو Delegate کرنا

```
Main agent: "payments module کی dependencies investigate کریں"
  -> Subagent (Explore): 15 files read □□ کرتا، imports trace □□ کرتا
  -> Returns: "Payments depends on AuthService, OrderModel, and the external
PaymentGateway API"

Main agent: context 15 میں files ایک □ کہ بجائے line □□ رکھتا
```

Separate context layer: Multi-agent systems □ر میں subagent محدود context budget — □□ میں کام کرتا □ کہ separate context layer ایک Coordinator ملتی □□□ information کے لیے □ ضروری task اسے □ صرف اپنا کام کرتا subagent outputs aggregate □□ کرتا، global state store اور □□ کرتا context allocate کرتا □□: وہ □ اس سے □□□ “context leakage” ایک □ہوتا، جہاں □نہیں agent window کو ایسی information جو □□ بھر دیتا □ سے دوسروں کے لیے □ غیر متعلق □ہو

Subagents □یا □ کے constrained context budgets:

- Minimal context بھیجیں: specific task + ضروری data
- Subagent کو raw data dumps □ بجائے □ کے structured results واپس کرنے □ کی □ہدایت دیں
- `allowedTools` □ سے subagent □ کے toolset کو limit کم — کریں tools کا مطلب کم distractions اور کم context cost

11.6 Structured State Persistence (crash recovery □یا □ کے)

□ر agent اپنی state ایک معلوم location پر export □□ کرتا:

```
// agent-state/web-search-agent.json
{
  "status": "completed",
  "queries_executed": ["AI music 2024", "AI music composition"],
  "results_count": 12,
  "key_findings": [...],
  "coverage": ["music composition", "music production"],
  "gaps": ["music distribution", "music licensing"]
}
```

Coordinator resume پر ایک manifest load □□ کرتا:

```
// agent-state/manifest.json
{
  "web-search": "completed",
  "doc-analysis": "in_progress",
  "synthesis": "not_started"
}
```

---

12 باب: Provenance کو محفوظ رکھنا

12.1 Attribution Loss Problem

جب متعدد sources سے results summarize ہوتے ہیں، تو یہ جاننا کہ “claim → source” link کون سا ہے گم ہو سکتا ہے:

```
Bad: "The AI music market is estimated at $3.2B." (No source, no year.)

Good:
{
  "claim": "The AI music market is estimated at $3.2B.",
  "source_url": "https://example.com/report",
  "source_name": "Global AI Music Report 2024",
  "publication_date": "2024-06-15",
  "confidence": 0.9
}
```

12.2 Conflicting Data کو Handle کرنا

جب دو sources مختلف values دیں:

```
{
  "claim": "Share of AI-generated music on streaming platforms",
  "values": [
    {
      "value": "12%",
      "source": "Spotify Annual Report 2024",
      "date": "2024-03",
      "methodology": "Automated classification"
    },
    {
      "value": "8%",
      "source": "Music Industry Association Survey",
      "date": "2024-07",
      "methodology": "Survey of 500 labels"
    }
  ],
  "conflict_detected": true,
  "possible_explanation": "Methodology اور time period کا فرق"
}
```

کسی ایک value کو چننے کے بجائے دونوں کو attribution کے ساتھ محفوظ رکھیں اور coordinator کو اندھا دھند منتخب نہ کریں۔ فیصلہ کرنے دیں۔

12.3 Correct Interpretation کے لیے Dates شامل کریں

Dates کے بغیر temporal differences کو contradiction سمجھ لیا جا سکتا ہے:

```
Bad: "Source A says 10%, source B says 15%. Contradiction."
Good: "Source A (2023) says 10%, source B (2024) says 15%. Likely 5+ ایک سال میں %ا
growth."
```

12.4 Content Type مطابق کر Render کریں

ہر چیز کو ایک ہی format میں نہ ٹھونسیں:

- Financial data -> tables
- News اور analysis -> prose
- Technical findings -> structured lists
- Time series -> chronological ordering

باب 13: Claude Code کے Built-in Tools

13.1 Tool Selection Reference

Task	Tool	Example
نام/pattern سے files تلاش کرنا	**Glob**	`**/*.test.tsx`, `src/components/**/*.ts`
File contents کے اندر search کرنا	**Grep**	Function name، error message، import
File مکمل پڑھنا	**Read**	Analysis کے لیے file load کرنا
نیا file بنانا	**Write**	Scratch سے file create کرنا
موجود file میں precise edit	**Edit**	Unique text match کے ذریعے specific snippet replace کرنا
Shell command چلانا	**Bash**	git، npm، tests، build

13.2 Incremental Investigation Strategy

ایک ساتھ تمام files بنائیں سمجھ بوجھ کو رفتہ رفتہ پڑھیں نہ:

```
1. Grep: entry points تلاش کریں (function definition، export)
2. Read: ملنے والی files پڑھیں
3. Grep: usages تلاش کریں (import، calls)
4. Read: consumer files پڑھیں
5. جب تک مکمل picture نہ مل جائے repeat کریں
```

13.3 Fallback: Edit بجائے کے Read + Write

جب Edit non-unique text match سے کی وجہ سے fail ہو جائے:

1. Read — مکمل file content لوڈ کریں
2. Content کو programmatically modify کریں
3. Write — updated version لکھ دیں

□حصہ II: امتحانی ڈومین نوٹس

---

1 ڈومین: Agent Architecture اور Orchestration (27%)

1.1 Autonomous Task Execution □لیے کے□ Agentic Loops Design کرنا

Key knowledge:

- Agent loop lifecycle: Claude request بھیجیں، `stop_reason` (`"tool_use"` بمقابل□ `"end_turn"`) چیک کریں، tools execute کریں، اگلی iteration □لیے کے□ results واپس کریں
- Tool results conversation history میں append □وتے□ □یں تاکہ□ model اگلا action □ط□ کر سکے
- Model-driven decision making (Claude اگلا tool □□ چنتا) بمقابل□ hard-coded decision trees

Key skills:

- Flow control: جب `stop_reason = "tool_use"` □و تو loop جاری رکھیں اور `"end_turn"` پر رک جائیں
- Iterations درمیان □کے tool results کو context میں append کرنا
- Anti-patterns بچنا □سے: completion □لیے کے□ assistant text parse کرنا، primary stopping mechanism □کے طور پر arbitrary iteration limits استعمال کرنا

1.2 Multi-agent Systems (Coordinator–Subagent) Orchestrate کرنا

Key knowledge:

- Hub-and-spoke architecture: coordinator تمام inter-agent communication، error handling، اور routing کا □□ مالک □وتا
- Subagents isolated context خود بخود □و — □یں coordinator کی history inherit □□یں کرتے□ میں کام کرتے□ □یں
- Coordinator responsibilities: task decomposition، delegation، result aggregation، dynamic selection of subagents
- Coordinator کی overly narrow decomposition □کا خطر□

Key skills:

- Research coverage کو subagents □تاکہ□ درمیان تقسیم کریں □کے duplication □و کم

- Iterative refinement loops implement کریں (coordinator synthesis evaluate اور کر tasks re-route کر)
- Observability تمام لیں کہ communication coordinator کے ذریعے کہ route کریں

1.3 Subagent Calls، Context Passing، اور Spawning Configure کرنا

Key knowledge:

- `Task` tool subagents spawn کرتا ہے؛ coordinator کے `allowedTools` میں `"Task"` شامل ہونا چاہیے
- Subagent context prompt میں explicitly شامل ہونا چاہیے؛ subagents parent context inherit نہیں کرتے
- `AgentDefinition` configuration: descriptions، system prompts، tool constraints
- Alternatives explore کرنے کے لیے کہ `fork_session` کے ذریعے کہ session management

Key skills:

- Previous agents کے full outputs subagent prompt میں شامل کریں
- Context pass کرتے وقت data اور metadata کو separate کرنے کے لیے کہ structured formats استعمال کریں
- ایک coordinator turn میں متعدد `Task` calls کے ذریعے کہ parallel subagents spawn کریں
- Coordinator prompts goals اور quality criteria طور پر لکھیں کہ، step-by-step instructions کے طور پر نہیں

1.4 Enforcement اور Handoff Patterns کے ساتھ Multi-step Workflows Implement کرنا

Key knowledge:

- Workflow ordering کے لیے کہ **programmatic enforcement** (hooks، preconditions) اور **prompt guidance** کے درمیان فرق
- جب deterministic guarantees ہوں درکار (مثلاً مالی operations پہلے سے identity verification)، prompts کافی نہیں ہوتے
- Escalation دوران کے structured handoff protocols (customer ID، reason، recommended action)

Key skills:

- ایسے programmatic preconditions جو downstream calls کو block کریں جب تک پہلے سے steps مکمل نہ ہوں (مثلاً `get_customer` کے verified ID تک واپس کرنے `process_refund` block کریں)
- Multi-aspect customer requests کو الگ items میں تقسیم کریں
- Human کو escalate کرتے وقت structured summaries produce کریں

1.5 Tool Calls Intercept اور کرنے Data Normalize کرنے کے لیے Agent SDK Hooks

Key knowledge:

- Hook patterns (مثلاً `PostToolUse`) tool results کو model کے consume پہلے سے کرنے intercept میں کرتے ہیں

- Hooks outgoing calls intercept کر کے compliance rules enforce کرتے ہیں (مثلاً threshold اوپر سے refunds block کرنا)
- Hooks **deterministic guarantees** دیتے ہیں، جبکہ prompt instructions **probabilistic compliance** دیتی ہیں

Key skills:

- `PostToolUse` hooks سے data formats normalize کریں (Unix timestamps، ISO 8601، numeric status codes)
- Policy-violating actions block کرنے کے لیے interception hooks اور استعمال کریں escalation کی طرف redirect کریں
- جب business rules کو guaranteed compliance چاہیے ہو تو prompts کے بجائے hooks منتخب کریں

1.6 Complex Workflows کے لیے Task Decomposition Strategies

Key knowledge:

- **Fixed pipelines** (prompt chaining) بمقابلہ intermediate results کی بنیاد پر **dynamic adaptive decomposition**
- Prompt chaining: sequential steps (ہر file کو الگ analyze کریں، پھر integration pass)
- Adaptive investigation plans جو discovered information کی بنیاد پر subtasks generate کرتے ہیں

Key skills:

- Predictable multi-aspect reviews کے لیے prompt chaining استعمال کریں؛ open-ended investigations کے لیے dynamic decomposition
- Large code reviews کو per-file analysis + separate cross-file integration pass میں تقسیم کریں
- Open-ended tasks کو degrade ہونے سے: پہلے structure map کریں، پھر prioritized plan بنائیں

1.7 Session State، Resuming، اور Forking

Key knowledge:

- Named sessions continue کرنے کے لیے `--resume <session-name>`
- Shared context سے independent investigation branches بنانے کے لیے `fork_session`
- Resume وقت agent کو file changes کی اہمیت بتانے
- Structured summary ساتھ نئی session کے ساتھ، stale results ساتھ resuming سے زیادہ reliable ہو سکتی ہے

Key skills:

- Named investigation sessions continue کرنے کے لیے `--resume` استعمال کریں
- Approaches parallel compare کرنے کے لیے `fork_session` استعمال کریں
- Resuming (context ابھی current ہے) اور نئی session شروع کرنے (results stale ہیں) کے درمیان انتخاب کریں

5.10 Session Management اور `fork_session`

`--resume <session-name>` named session کو resume □□ کرتا:

```
claude --resume investigation-auth-bug
```

- saved context کے ساتھ پچھلی conversation □□ جاری رکھتا
- Long investigations جو متعدد sessions پر پھیلی □وں ان کے لیے مفید
- Risk: اگر پچھلی session بعد کے files تو □وں بدل چکی tool results stale □یں □و سکتے

`fork_session` shared context سے independent branch □□ بناتا:

```
Codebase investigation
         |
    fork_session
    /           \
Approach A:      Approach B:
Redux            Context API
```

- دونوں forks branch point تک context inherit □یں کرتے
- diverge □یں کے بعد اس وہ آزادانہ طور پر کرتے
- Approaches compare یا کرنے strategies test مفید کے لیے

Resume نئی □ بجائے کے session کب شروع کریں:

- Tool results stale □وں (files □وں بدل چکی)
- context degrade □و گیا □و اور کافی وقت گزر چکا □و
- پرانے tool data کے ساتھ کے resume □□: "□م نے جو پایا اس کا مختصر خلاصہ یہ □□ کرنے کے بجائے یہ بہتر □□ کہ سے "... restart کیا جائے

---

2 ڈومین: Tool Design اور MCP Integration (18%)

2.1 Clear Descriptions ساتھ کے Tool Interfaces Design کرنا

Key knowledge:

- Tool descriptions **بنیادی** وہ **mechanism** ہے جس سے LLM tools select □□ کرتا؛ minimal descriptions unreliable selection □یں دیتی
- Input formats، example queries، edge cases، اور applicability boundaries □امیت شامل کرنے کی
- Ambiguous یا overlapping descriptions misrouting □یں کا سبب بنتی
- System prompt wording tools ساتھ غیر ارادی کے associations □□ بنا سکتی

Key skills:

- ایسی descriptions □ر لکھیں جو tool کو similar alternatives پر واضح طور سے distinguish کریں

- Functional overlap کو ختم کرنے کے لیے tools rename کریں (مثلاً `analyze_content` -> `extract_web_results`)
- General-purpose tools کو specialized tools میں تقسیم کریں جن کے input/output contracts واضح ہوں

2.2 MCP Tools کے لیے Structured Error Responses Implement کرنا

Key knowledge:

- MCP tool responses میں `isError` flag
- **Transient errors** (timeouts)، **validation errors** (bad input)، **business errors** (policy violations)، اور **access/permission errors** کے درمیان فرق
- Generic errors ("Operation failed") صحیح recovery decisions میں روک دیتے ہیں
- Retryable اور non-retryable errors میں فرق

Key skills:

- `errorCategory` (transient/validation/permission)، `isRetryable`، اور human-readable message جیسی structured metadata واپس کریں
- Business-rule violations واضح کرنے کے لیے user-facing explanations کے ساتھ `retryable: false` استعمال کریں
- Transient failures کے لیے subagents میں local recovery کریں؛ صرف وہ errors propagate کریں جو وہ خود حل نہ کر سکیں
- Access failures (retry decision) اور valid empty results (no matches) میں فرق کریں

2.3 Agents میں Tools Allocation اور `tool_choice` Configure کرنا

Key knowledge:

- 4–5 tools (مثلاً 18 کے بجائے) tool selection reliability کو کم کرتی ہے
- Specialization ہر subagent کے tools کو agents رکھنے والے ہیں غلط استعمال کرتے ہیں
- Scoped tool access: role-relevant tools + محدود cross-role utilities
- `tool_choice`: `"auto"`, `"any"`, اور forced tool selection (`{"type": "tool", "name": "..."}`)

Key skills:

- ہر subagent کا toolset صرف relevant tools تک محدود رکھیں
- General tools کو constrained alternatives سے بدلیں (مثلاً `fetch_url` -> `load_document`)
- `tool_choice: "any"` استعمال کریں تاکہ text answer کے بجائے یقینی tool call ہو
- Specific tool force کریں تاکہ execution order assured ہو

2.4 MCP Servers کو Claude Code اور Agent Workflows میں Integrate کرنا

Key knowledge:

- MCP server scope: □□ ٹیموں کے project (`.mcp.json`) بمقابلہ□□ experiments □□ کے user (`~/.claude.json`)
- Secrets management □□ کے `.mcp.json` میں environment variable substitution (مثلاً `${GITHUB_TOKEN}`)
- تمام connected MCP servers کے tools connection پر discover ایک ساتھ اور □□یں □□ جاتے□□ available □□وتے□□یں
- MCP resources بطور “content catalogs” (task summaries، database schemas) exploratory tool calls کم کرتے□□یں

Key skills:

- Shared MCP servers کو project `.mcp.json` میں env-var-based tokens ساتھ کے□□ configure کریں
- Personal/experimental servers کو `~/.claude.json` میں رکھیں
- Standard integrations □□ کے□□ community MCP servers کو custom servers پر ترجیح دیں

2.5 Built-in Tools (Read, Write, Edit, Bash, Grep, Glob) منتخب اور Apply کرنا

Key knowledge:

- **Grep**: file contents اندر کے□□ search (function names، error messages، imports)
- **Glob**: name/extension patterns □□ سے files تلاش کرنا
- **Read/Write**: full-file operations؛ **Edit**: unique text matches □□ ذریعے□□ کے precise changes
- اگر Edit non-unique matches □□ سے□□ کی وجہ سے fail تو□□ Read + Write پر واپس جائیں

Key skills:

- Content search □□ کے□□ Grep اور pattern-based discovery □□ کے□□ Glob استعمال کریں
- سمجھ بوجھ رفتہ رفتہ بنائیں: entry points □□ کے□□ Grep، پھر flows trace □□ کے□□ کرنے Read
- Wrapper modules □□ ذریعے□□ کے function usage trace کریں

ڈومین 3: Claude Code Configuration اور Workflows (20%)

3.1 Hierarchy، Scope، اور Modular Organization کے ساتھ CLAUDE.md Configure کرنا

Key knowledge:

- CLAUDE.md hierarchy: user (`~/.claude/CLAUDE.md`)، project (`.claude/CLAUDE.md` یا root `CLAUDE.md`)، اور directory-level (subdirectories میں CLAUDE.md)
- User-level settings صرف ایک user پر لاگو ہوتی ہیں اور VCS کے ذریعے share نہیں ہوتی
- External files refer کرنے کے لیے `@path` syntax (مثلاً `@./standards/coding-style.md`) تاکہ CLAUDE.md modular بنے
- Monolithic CLAUDE.md بجائے topic-focused rule files کے لیے `.claude/rules/` directory

Key skills:

- Hierarchy issues diagnose کریں (نئی team member کو instructions وہی ملتیں کیونکہ وہ user-level میں تھیں، project-level میں نہیں)
- ہر package کے CLAUDE.md میں standards selectively include کرنے کے لیے `@path` (مثلاً `@./standards/testing.md`) استعمال کریں
- بڑی CLAUDE.md کو متعدد `.claude/rules/` files (testing.md، api-conventions.md، deployment.md) میں تقسیم کریں

3.2 Custom Slash Commands اور Skills بنانا اور Configure کرنا

Key knowledge:

- `.claude/commands/` میں **Project commands** (VCS کے ذریعے shared) بمقابلہ `~/.claude/commands/` میں **user commands**
- `.claude/skills/` میں SKILL.md frontmatter: `context: fork`، `allowed-tools`، `argument-hint`
- `context: fork` skill کو isolated subagent context میں چلاتا ہے تاکہ main session pollute نہ ہو
- Personal skill variants `~/.claude/skills/` میں مختلف ناموں سے رکھی جا سکتی ہیں

Key skills:

- Project slash commands `.claude/commands/` میں رکھیں تاکہ پوری team انہیں حاصل کر سکے
- Verbose output والے skills isolate کرنے کے لیے `context: fork` استعمال کریں
- Skill کے tools محدود کرنے کے لیے `allowed-tools` استعمال کریں
- Required parameters مانگنے کے لیے `argument-hint` استعمال کریں

3.3 Conditional Convention Loading کے لیے Path-specific Rules استعمال کرنا

Key knowledge:

- `.claude/rules/` files میں YAML frontmatter `paths` glob patterns شامل ہو سکتا ہے تاکہ مطابق rules activate ہوں
- Path-scoped rules صرف matching files edit کرتے وقت load ہوتے ہیں، context اور tokens بچتے ہیں
- Glob-based path rules directory-level CLAUDE.md سے بہتر ہو سکتے ہیں جب conventions کئی directories میں apply ہوں (مثلاً tests)

Key skills:

- `paths: ["terraform/**/*"]` کے ساتھ `.claude/rules/` files صرف بنائیں تاکہ matching files پر load ہوں
- Glob patterns (`**/*.test.tsx`) استعمال کریں تاکہ location قطع نظر file type سے لحاظ سے conventions apply ہوں
- جب conventions codebase تو ہوں بھر میں پھیلی directory-level CLAUDE.md بجائے کے path-specific rules کو ترجیح دیں

3.4 Planning Mode بمقابلہ Direct Execution کب استعمال کریں

Key knowledge:

- **Planning mode**: complex tasks، بڑے changes، multiple viable approaches، اور architectural decisions کے لیے
- **Direct execution**: سادہ، اچھی طرح سمجھے گئے changes (مثلاً single validation شامل کرنا)
- Planning mode changes پر سے codebase کی safe exploration کو ممکن بناتا ہے
- Explore subagent verbose discovery output isolate کرتا ہے

Key skills:

- Architectural consequences والے tasks (microservices، 45+ files والی migrations) کے لیے planning mode استعمال کریں
- واضح stack trace اور single file والے fixes کے لیے direct execution استعمال کریں
- Multi-phase tasks میں context-window exhaustion کو روکنے کے لیے Explore subagent استعمال کریں
- Approaches کو combine کریں: discovery کے لیے plan، implementation کے لیے execute

3.5 Progressive Improvement کے لیے Iterative Refinement

Key knowledge:

- Concrete input/output examples expectations communicate کرنے کا سب سے مؤثر طریقہ ہیں
- **Test-driven iteration**: پہلے tests لکھیں، پھر failures کی بنیاد پر iterate کریں
- “Interview” pattern: Claude clarifying questions پوچھتا ہے تاکہ non-obvious design considerations سامنے آئیں

- تمام issues ایک message میں کب دیں (interdependent) اور sequentially کب (independent)

Key skills:

- Transformation requirements 3–2 کے لیے واضح کرنے کے concrete input/output examples فراہم کریں
- Implementation سے پہلے expected behavior، edge cases، اور performance requirements کے ساتھ test sets بنائیں
- Design aspects (cache invalidation، failure modes) کے لیے سامنے لانے کے interview pattern استعمال کریں
- Edge cases کے لیے sample inputs اور expected outputs کے ساتھ concrete test cases دیں

3.6 Claude Code کو CI/CD Pipelines میں Integrate کرنا

Key knowledge:

- Automated pipelines میں non-interactive mode کے لیے `-p` (یا `--print`) flag
- CI میں structured output کے لیے `--output-format json` اور `--json-schema`
- CLAUDE.md project context (testing standards، review criteria) کو فراہم کرتا ہے
- **Session context isolation**: code generate کرنے والی same session اسے review کرنے میں کم مؤثر ہوتی ہے

Key skills:

- Interactive input پر hang ہونے سے بچنے کے لیے CI میں Claude Code کو `-p` کے ساتھ چلائیں
- Structured results (مثلاً inline PR comments) کے لیے `--output-format json` + `--json-schema` استعمال کریں
- نئے commits کے بعد دوبارہ run کرتے وقت پچھلے review results شامل کریں (صرف new/unfixed issues report کریں)
- Test generation quality بہتر کرنے کے لیے CLAUDE.md میں testing standards اور available fixtures document کریں
- نئے tests generate کرتے وقت existing test files کو context میں شامل کریں تاکہ duplication سے بچا جا سکے اور style consistent رہے

ڈومین 4: Prompt Engineering اور Structured Output (20%)

4.1 Accuracy بہتر کرنے کے لیے Explicit Criteria کے ساتھ Prompts Design کرنا

Key knowledge:

- Explicit criteria vague instructions سے زیادہ مؤثر ہوتے ہیں (مثلاً “flag comments only when they contradict code” بمقابلہ “check comment accuracy”)
- “be more conservative” جیسی generic guidance concrete categorical criteria سے کم مؤثر ہے

- False positives developer trust کچھ: کیں ہوتا پر اثر انداز categories میں high false-positive rates درست کیں کم کر دیتی trust پر بھی categories

Key skills:

- Review criteria define کیا: کریں report نہ کرنا (bugs، security) اور کیا ignore نہ کرنا (minor style)
- High false-positive rates والی categories پر عارضی طور disable کریں
- ہر level لیے کے لیے code examples ساتھ کے explicit severity criteria define کریں

4.2 Output Consistency بہتر کرنے کے لیے Few-shot Prompting استعمال کرنا

Key knowledge:

- Few-shot examples consistently formatted، actionable output مؤثر کا سب سے کرنے پیدا method میں
- Few-shot مبہم cases (tool selection، test coverage gaps) کو دکھا سکتا ہے
- Few-shot model کو نئے patterns پر generalize کرنے میں مدد دیتا ہے، صرف defaults دہرانے میں نہیں
- Few-shot extraction tasks میں hallucinations کم کر سکتا ہے

Key skills:

- Ambiguous scenarios کے لیے کے ساتھ rationale 2–4 کے targeted examples دیں
- Output format (location، issue، severity، suggested fix) دکھانے والے examples شامل کریں
- ایسے examples دیں جو acceptable code patterns اور real issues میں فرق دکھائیں
- مختلف structures والے documents سے درست extraction کے examples دیں

4.3 `tool_use` اور JSON Schemas کے ساتھ Structured Output Enforce کرنا

Key knowledge:

- `tool_use` with JSON Schemas schema-conformant output اور کی ضمانت دینے JSON syntax errors ختم کرنے کا سب سے reliable طریقہ ہے
- `tool_choice: "auto"` میں model text واپس کر سکتا ہے؛ `"any"` لازماً میں سے tool call کرنی ہوتی ہے؛ forced selection specific tool منتخب کرتی ہے
- Strict JSON Schemas syntax errors ختم کرتی ہیں مگر semantic errors نہیں روکتیں (totals نہیں ملتے؛ values غلط fields میں)
- Schema design: required vs optional fields؛ enums ساتھ کے “other” + detail string extensibility کے لیے

Key skills:

- JSON Schemas ساتھ کے extraction tools define کریں اور `tool_use` results سے data parse کریں
- متعدد schemas میں تو structured output یقینی بنانے کے لیے `tool_choice: "any"` استعمال کریں
- Specific tool call force کریں: `tool_choice: {"type": "tool", "name": "extract_metadata"}`
- Source میں information کی صورت میں نہ ہونے values fabricate کرنے سے بچانے کے لیے fields optional/nullable رکھیں

- Extensible categorization کے لیے `"other"` اور `"unclear"` جیسے enum values + detail fields استعمال کریں

4.4 Extraction Quality کے لیے Validation، Retries، اور Feedback Loops Implement کرنا

Key knowledge:

- Retry-with-error-feedback: correction guide کے لیے کرنے retry prompt میں concrete validation errors شامل کریں
- اگر information source تو نے موجود نہیں کیا retries میں وہ فائدہ نہیں ہوتا
- Feedback loop design: finding کو trigger والا کرنے pattern (`detected_pattern`) track کریں
- Semantic errors (totals reconcile نہیں ہوتے) بمقابلہ syntax errors (جو `tool_use` سے address ہوتے ہیں)

Key skills:

- Original document، incorrect extraction، اور specific validation errors ساتھ کے follow-up prompts دیں
- جب retry سے فائدہ ہو (required info صرف external document میں ہو) تو اس کی شناخت کریں
- False positives analyze کرنے کے لیے findings میں `detected_pattern` fields شامل کریں
- Discrepancies detect کرنے کے لیے `stated_total` اور `calculated_total` دونوں extract کر کے self-correction design کریں

4.5 Efficient Batch Processing Strategies Design کرنا

Key knowledge:

- Message Batches API: 50% savings، 24-hour تک processing window، latency SLA guarantee نہیں
- Batch processing non-blocking tasks (overnight reports، audits) اور مناسب ہے جبکہ blocking tasks (pre-merge checks) کے لیے نہیں
- Batch API ایک single request میں multi-turn tool calling support نہیں کرتا
- `custom_id` fields batch اندر کے request/response correlate کرتی ہیں

Key skills:

- Blocking checks کے لیے synchronous API اور overnight/weekly workloads کے لیے Batch API استعمال کریں
- SLA needs مطابق batch submission cadence plan کریں (مثلاً 30-hour guarantee 24 کے لیے 4-hour windows ساتھ کے 4-hour processing)
- Failed documents (identified by `custom_id`) صرف انہی کو دوبارہ submit کر کے failures handle کریں
- Large-scale processing پر جانے سے sample ساتھ کے prompts iterate کریں

4.6 Multi-instance اور Multi-pass Review Architectures Design کرنا

Key knowledge:

- Self-review limitations: model اپنی reasoning context ساتھ رکھتا ہے اور اپنے فیصلوں کو challenge کرنے کا امکان کم ہوتا ہے
- Independent review instances (generation context بغیر کے) subtle issues میں بہتر ہوتی ہیں تلاش کرنے میں
- Multi-pass review: per-file local analysis + cross-file integration pass attention dilution سے بچاتا ہے

Key skills:

- Changes review کرنے کے لیے generation context بغیر کے second independent Claude instance استعمال کریں
- Multi-file reviews کو per-file passes + integration passes میں تقسیم کریں تاکہ cross-file dataflow analysis ہو سکے
- Self-rated confidence ساتھ کے verification passes استعمال کر کے reviews کو calibrated طریقے سے route کریں

ڈومین 5: Context Management اور Reliability (15%)

5.1 Critical Information محفوظ رکھنے کے لیے Conversation Context Manage کرنا

Key knowledge:

- Progressive summarization خطرات کے: numeric values، percentages، اور dates vague summaries میں بدل جاتے ہیں
- Lost-in-the-middle effect: models long inputs کے شروع اور آخر کو زیادہ reliable طریقے سے process کرتے ہیں، لیکن درمیان کے findings miss کر سکتے ہیں
- Tool outputs relevance میں مقابلے کے context میں disproportionately جمع ہو سکتے ہیں (40+ fields 5 جب درکار ہوں)
- Subsequent API requests میں full conversation history کی ضمنیت بھیجنے

Key skills:

- Transactional facts کو summarized history ایک باہر سے persistent “case facts” block میں نکالیں
- Verbose tool outputs کو relevant fields تک trim کریں
- Key findings کو aggregated data شروع میں کے explicit section headings ساتھ رکھیں کے
- Subagents سے structured outputs میں metadata (dates، sources) شامل کروائیں

5.2 Effective Escalation Patterns Design کرنا اور Ambiguity Resolve کرنا

Key knowledge:

- Suitable escalation triggers: human کی explicit request، policy gaps/exceptions، پیش رفت نہ ہو پانا
- Immediate escalation (explicit request) بمقابلہ attempt-to-resolve (agent scope اندر کہ)
- Sentiment analysis اور model confidence self-ratings case complexity کہ unreliable proxies ہیں
- Multiple customer matches ہیں کہ heuristic guessing بجائے کہ additional identifiers مانگنے چاہئیں

Key skills:

- System prompt میں explicit escalation criteria few-shot examples دیں کہ ساتھ
- Human کی explicit request کو بغیر اضافی investigation فوراً کہ execute کریں
- جب policy کسی specific request ہیں کہ ambiguous یا silent ہو تو escalate کریں
- Tool results میں multiple matches ہوں تو additional identifiers مانگیں

5.3 Multi-agent Systems میں Error Propagation Strategies Implement کرنا

Key knowledge:

- Structured error context (failure type، query، partial results، alternatives) coordinator کو smarter recovery دیتا ہے
- Access failures (timeouts retry decision ہیں چاہتے ہیں) اور valid empty results (no matches) میں فرق کریں
- Generic error statuses (“search unavailable”) coordinator سے valuable context ہیں چھپا دیتے ہیں
- Silent suppression یا ایک failure پر پورا workflow abort کرنا دونوں anti-patterns ہیں

Key skills:

- Structured error context واپس کریں: failure type، کیا attempt کیا گیا، partial results، possible alternatives
- Access failures کو valid empty results سے الگ کریں
- Transient failures ہیں کہ subagents میں local recovery صرف کریں؛ non-recoverable errors partial results ساتھ کہ propagate کریں
- Synthesis میں coverage annotate کیا اچھی طرح کریں: supported کہاں اور gaps ہیں باقی

5.4 Large Codebases Investigate کرتے وقت Context Efficiently Manage کرنا

Key knowledge:

- Long sessions میں context degradation: model unstable answers اور مخصوص ہے لگتا ہے دینے classes کہ بجائے “typical patterns” کا حوالہ دیتا ہے
- Scratchpad files key findings کو context boundaries میں کہ پار محفوظ رکھتے ہیں
- Subagents کو delegate سے کرنے verbose discovery output isolate ہے ہو جاتی

- Structured state persistence crash recovery □□ ممکن بناتی

Key skills:

- Specific questions □لیے □کہ subagents spawn □کریں جبکہ high-level coordination main agent میں رکھیں
- Key findings میں بعد اور محفوظ رکھنے□ refer □لیے □کہ کرنے□ scratchpad files استعمال کریں
- □اگلا phase □کہ subagents spawn □سے پہلے□ کرنے□ key findings summarize کریں
- Long investigations دوران □کہ context usage □لیے □کہ کرنے□ کم `/compact` استعمال کریں

5.5 Human Oversight اور Confidence Calibration کے□ ساتھ Workflows Design کرنا

Key knowledge:

- Aggregate metrics (97 مثلاً% overall accuracy) specific document types یا fields □پر کمزوری چھپا سکتے□یں
- Stratified random sampling high-confidence extractions میں error rates measure □□ کرتا
- Field-level confidence calibration labeled validation sets □کے ذریعے
- Automation □پہلے□ سے document type اور field segment □کے حساب سے□ accuracy validate کریں

Key skills:

- New error patterns detect □لیے □کہ کرنے□ stratified random sampling implement کریں
- Stable performance validate □لیے □کہ کرنے□ document type اور field □کے حساب سے□ accuracy analyze کریں
- Field-level confidence scores output اور کریں labeled data □کے ذریعے□ review thresholds calibrate کریں
- Low-confidence یا ambiguous-source extractions human review کی طرف route کریں

5.6 Multi-source Synthesis میں Provenance Preserve کرنا اور Uncertainty Handle کرنا

Key knowledge:

- Summarization دوران □کہ attribution اگر □□ کھو جاتی “claim → source” mappings محفوظ نہ □یں
- Aggregation دوران □کہ structured mappings برقرار رکھنی چاہ□ئیں
- Conflicting statistics کو arbitrarily ایک value □بجائے□ کہ□ منتخب کرنے□ attribution ساتھ □کہ conflict annotate کر □کہ handle کریں
- Temporal differences کو contradiction □لیے □کہ بچنے□ سے□ سمجھنے□ publication/collection dates شامل کریں

Key skills:

- Subagents □سے “claim → source” mappings (URL، document name، quotes) output کروائیں
- Reports کو stable findings اور disputed ones میں الگ structure کریں
- Conflicting values کو annotations اور محفوظ رکھیں ساتھ □کہ reconciliation □لیے □کہ coordinator کو دیں
- Correct temporal interpretation □لیے □کہ publication dates شامل کریں

- Content type مطابق کے render کریں: financial data tables میں، news prose میں، technical findings structured lists میں

Claude Certified Architect — Foundations Certification

مطالعہ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

Claude Certified Architect — Foundations سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر Claude-based حل بنانے وقت درست trade-off امتحان یہ فیصلہ کر سکتا ہے Claude Code، Claude Agent SDK، اور Model Context Protocol (MCP) جن کی بنیادی علم کو جانچتا ہے — یعنی وہ بنیادی ٹیکنالوجیز Claude API، کے ساتھ پروڈکشن ایپلیکیشنز تیار کی جاتی ہیں جو Claude سے

امتحان کے سوالات حقیقی دنیا کے منظرناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے agentic systems بنانا، developer productivity میں شامل کرنا CI/CD کو Claude Code ڈیزائن کرنا، multi-agent research pipelines structured data نکالنا، اور غیر ساختہ دستاویزات سے tools بنانا

ہدف امیدوار

مثالی امیدوار ایک **solution architect** جو Claude اور ڈیزائن ایپلیکیشنز پروڈکشن ساتھ کے ship آپ کرتا ہے کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- **Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- **Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- **Model Context Protocol (MCP)** — backend integration کے لیے tools اور resources
- **Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- **Context windows** — لمبی دستاویزات، multi-agent context passing
- **CI/CD pipelines** — automated code review، test generation
- **Escalation and reliability** — error handling، human-in-the-loop

---

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (4 میں سے 1 درست)

اسکورنگ	100–1000 scale، passing score **720**
Guessing penalty	نہیں (ہر سوال کا جواب دیں!)
Scenarios	8 4 میں سے (randomly selected)

امتحانی مواد: 5 ڈومینز

ڈومین	وزن
1. Agent architecture and orchestration	**27%**
2. Tool design and MCP integration	**18%**
3. Claude Code configuration and workflows	**20%**
4. Prompt engineering and structured output	**20%**
5. Context management and reliability	**15%**

امتحانی منظرنامہ

Scenario 1: Customer Support Agent

آپ ایک agent بنا رہے ہیں جو Claude Agent SDK استعمال کرتے ہوئے returns، billing disputes، اور account issues سنبھالتا ہے۔ agent MCP tools (`get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`) 80 فیصد+ first-contact resolution مناسب وقت، escalation کے ساتھ حاصل کرتا ہے۔

Scenario 2: Claude Code کے ساتھ Code Generation

آپ Claude Code کو development میں استعمال کرتے ہیں تیز کرنے کے لیے: code generation، refactoring، debugging، documentation۔ آپ کو اس کے CLAUDE.md configuration اور custom slash commands کے ساتھ integrate کرنا، planning mode کے استعمال کو سمجھنا

Scenario 3: Multi-Agent Research System

ایک coordinator agent specialized subagents کو tasks سونپتا ہے: web research، document analysis، synthesis، اور report generation۔ سسٹم کو citations کے ساتھ مکمل رپورٹس تیار کرنی چاہئیں

Scenario 4: Developer Productivity Tools

یہ agent engineers کو unfamiliar codebases explore کرنے، boilerplate code اور routine tasks بنانے، اور MCP servers اور Built-in tools (Read، Write، Bash، Grep، Glob) استعمال کرنے میں مدد دیتا ہے۔ یہ automate ہوتے ہیں

Scenario 5: Claude Code for Continuous Integration

Claude Code کو CI/CD pipeline میں ضم کریں تاکہ automated code reviews، test generation، اور pull request feedback حاصل ہو۔ Prompts کو ایسے بننے چاہئیں کہ false positives کم سے کم ہوں۔

Scenario 6: Structured Data Extraction

سسٹم غیر ساختہ دستاویزات سے معلومات نکالتا ہے، output کو JSON schemas کے ذریعے validate کرتا ہے، درست طور پر سنبھالنے چاہئیں edge cases برقرار رکھتا ہے اور high accuracy ہے۔

Scenario 7: Conversational AI Architecture Patterns

آپ multi-turn conversational systems میں جن میں ڈیزائن کرتے ہیں context window management، متعدد turns میں instructions کا برقرار رہنا، memory strategies، محفوظ execution کے لیے tool design، اور مبہم یا متضاد user inputs کو سنبھالنا شامل ہے۔

Scenario 8: Agentic AI Tools *(مواد موجود نہیں — ہماری مدد کریں!)*

اس scenario والا حصہ ابھی تک candidates کی اطلاع امتحان دینے guide میں شامل نہیں ہے لیکن یہ دی ہوئی ہے۔ اگر آپ نے اصل امتحان میں اس scenario کے سوالات دیکھے ہیں، تو انہیں GitHub Issues میں share کریں تاکہ ہم مکمل coverage تیار کر سکیں۔ آپ کا تعاون ہر اس شخص کی مدد کرے گا جو امتحان کی تیاری کر رہا ہے۔

حصہ I: نظریاتی بنیادیں

یہ حصہ وہ بنیادی نظریے بیان کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے۔ مواد کو topic کے مطابق — اس سے ہر domains کے مطابق ترتیب دیا گیا ہے، نہ کہ امتحانی concepts اور technologies کی گہری سمجھ بنتی ہے۔

---

1 باب: Claude API — Model Interaction کی بنیادیں

> دستاویزات: Messages API | Prompt Engineering

1.1 API Request Structure

Claude API ایک request–response model ہے۔ Claude Messages API request میں عموماً یہ شامل ہوتا ہے:

```
{
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "system": "You are a helpful assistant.",
    "messages": [
        {"role": "user", "content": "Hi!"},
        {"role": "assistant", "content": "Hello!"},
        {"role": "user", "content": "How are you?"}
    ],
    "tools": [...],
    "tool_choice": {"type": "auto"}
}
```

اہم فیلڈز:

- `model` — model selection (`claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5`)
- `max_tokens` — response میں زیادہ سے زیادہ tokens
- `system` — system prompt (model کی تعریف کردہ رویہ)
- `messages` — conversation history (**بھیجنا ضروری** **full history**)
- `tools` — available tools کی definitions
- `tool_choice` — tool selection strategy

1.2 Message Roles

`messages` array دو conversational roles اور ایک instructional role استعمال کرتا ہے:

- `user` — user messages، بشمول tool results (ایک `user`-role message اندر کردہ `tool_result` content block، ایک الگ کی صورت میں بھیجے جاتے ہیں، `tool` role طور پر نہیں کردہ)
- `assistant` — model responses (history میں محفوظ شامل وقت بھیجتے)، بشمول tool use requests (`tool_use` content blocks)
- `system` — top-level `system` field کے ذریعے سیٹ کیا جا سکتا ہے (یا سے لاگو ہوتا ہے turn پہلے)، اس نقطے سے آگے شامل کیا جا سکتا ہے inline طور پر کردہ `{"role": "system", ...}` میں `messages` مخصوص، ہوتا لاگو placement rules دیکھیں نیچے — تابع کردہ)

Tool results کو `"tool"` role والے message ایک ایک طور پر نہیں بھیجا جاتا ہے بلکہ `user`-role message طور پر کردہ شامل ہوتا ہے content block `tool_result` میں ایک content پر بھیجے جاتے ہیں جس کردہ:

```
{
  "role": "user",
  "content": [
    {
      "type": "tool_result",
      "tool_use_id": "toolu_01...",
      "content": "..."
    }
  ]
}
```

`system` براہ راست `messages` array میں بھی ایک role صرف نہ ہو سکتا ہے، نہ top-level طور پر ظاہر ہو سکتا ہے کہ `system` parameter اس کا مقصد ذریعے top-level `system` field کے cached prefix کو invalidate بغیر کیے شامل کرنا ہے۔ اس کے مخصوص instructions کے درمیان placement rules یہ ہیں:

- ایک `user` turn (بشمول وہ جس میں `tool_result` blocks ہوں) یا server tool use پر ختم ہونے والا `assistant` turn فوراً بعد آنا چاہیے
- کسی `assistant` turn یا پہلا آنا چاہیے یا array سے آخری element ہونا چاہیے
- کسی `tool_use` block اور اس کے `tool_result` 400 پر ایسا کرنے — آ سکتا درمیان نہیں کہ error ملتا ہے
- پہلا والوں پر اور بعد (کے درمیان شامل کیے گئے conversation بشمول) `system` messages بعد میں آنے والا top-level `system` field پر فوقیت رکھتے ہیں کے لیے کے turns

اہم بات: ہر API request میں مکمل conversation history بھیجیں Model requests کے درمیان کے state محفوظ آزاد ہوتی ہیں call نہیں رکھتا؛ ہر

1.3 `stop_reason`

Claude API response میں `stop_reason` کے ہوتا ہے جو بتاتا ہے کہ model کیوں رُکا:

Value	Description	Action
`"end_turn"`	Model نے جواب مکمل کر لیا	user کو نتیجہ دکھائیں
`"tool_use"`	Model tool call کرتا ہے	tool چلائیں اور result واپس دیں
`"max_tokens"`	Token limit ختم ہو گئی	response truncated؛ limit بڑھا سکتے ہیں
`"stop_sequence"`	Stop sequence مل گئی	application logic کے مطابق handle کریں

Agentic systems میں `"tool_use"` اور `"end_turn"` سب سے اہم signals ہیں — یہی loop control کرتے ہیں

1.4 System Prompt

System prompt ایک خاص instruction جو ہے context اور behavioral rules ہے یہ ہیں متعین کرتا:

- `messages` array الگ؛ ہوتا نہیں حصہ کا `system` field ہے میں دیا جاتا
- user messages ہے پر فوقیت رکھتا
- ہو کر پوری گفتگو میں لاگو رہتا ہے ایک بار load
- role، constraints، اور output format ہے ہوتا استعمال کے لیے کی وضاحت

امتحان کے لیے اہم: system prompt کی wording tool selection مثال کو غیر ارادی طور پر متاثر کر سکتی ہیں ضرورت سے زیادہ `get_customer` کو model کی تصدیق کریں" جیسی ہدایت کے customer طور پر "ہمیشہ استعمال کرنے پر لے جا سکتی ہیں

1.5 Context Window

Context window میں کل متن (tokens) جس کو model ایک وقت میں process کر سکتا ہے اس میں شامل ہے:

- System prompt
- مکمل message history
- Tool definitions
- Tool results

اہم مسائل:

1. **Lost-in-the-middle effect:** لمبے input کی درمیان میں، آغاز اور اختتام کی معلومات زیادہ مؤثر ہوتی ہیں
   حل: اہم معلومات شروع یا آخر میں رکھیں details miss ہو سکتی ہیں
2. **Tool results کا جمع ہونا:** ہر tool call context میں output اگر 40+ fields واپس شامل کرتی ہے ضائع ہوتا ہے context کرہ مگر صرف 5 اہم ہوں تو
3. **Progressive summarization:** history compress پر کرنے numeric values، percentages، اور dates اکثر vague ہو جاتی ہیں

باب 2: Tools اور `tool_use`

> دستاویزات: Tool Use

2.1 `tool_use` کیا ہے

`tool_use` وہ mechanism ہے ذریعے جس کو Claude external functions call کرتا ہے Model کو براہ
واپس کرتا result کر کو execute اسے code بناتا ہے؛ آپ کا structured tool call نہیں چلاتا — وہ code راست
ہے

2.2 Tool Definition

ہر tool JSON schema کے ذریعے کو define ہوتا ہے:

```
{
    "name": "get_customer",
    "description": "Finds a customer by email or ID. Returns the customer profile,
including name, email, order history, and account status. Use this tool BEFORE
lookup_order to verify the customer's identity. Accepts an email (format:
user@domain.com) or a numeric customer_id.",
    "input_schema": {
        "type": "object",
        "properties": {
            "email": {"type": "string", "description": "Customer email"},
            "customer_id": {"type": "integer", "description": "Numeric customer ID"}
        },
        "required": []
    }
}
```

Tool description کے اہم نکات:

1. Description ہی tool selection کا بنیادی mechanism ہے۔ Minimal descriptions غلط selection کا سبب بنتی ہیں۔
2. Description میں یہ شامل کریں: tool کیا کرتا ہے، کیا واپس کرتا ہے، input format کیا ہے، edge cases کیا ہیں، اور یہ کب استعمال ہو۔
3. Overlapping descriptions سے بچیں۔
4. Built-in tools اور MCP tools میں overlap ہو تو MCP tool کو بہتر اور واضح بنائیں۔

2.3 `tool_choice`

Value	Behavior	کب استعمال کریں
`{ "type": "auto" }`	Model خود طے کرتا ہے کہ tool call کرنی ہے یا نہیں	عام default
`{ "type": "any" }`	Model کو لازماً کوئی tool call کرنی ہوگی	جب structured output چاہیے
`{ "type": "tool", "name": "extract_metadata" }`	لازماً خاص tool call ہوگی	forced first step / order کے لیے

2.4 JSON Schemas for Structured Output

JSON schemas کے ساتھ `tool_use` استعمال کرنا Claude سے structured output لینے کا سب سے reliable طریقہ ہے۔ یہ syntax errors اور required structure کم کرتا ہے، مگر semantic correctness کی ضمانت نہیں دیتا۔

Design اصول:

1. صرف وہ fields required رکھیں جو ہمیشہ ملنی چاہئیں
2. ممکنہ missing data کے لیے nullable fields استعمال کریں
3. `"other"` اور `"unclear"` جیسے enum values شامل کریں تاکہ information ضائع نہ ہو

2.5 Syntax اور Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON، غلط field type	JSON schema ساتھ کے `tool_use`
Semantic	Totals match نہیں کرتے، value غلط field میں	Validation checks، feedback retry

---

باب 3: Claude Agent SDK — Agentic Systems بنانا

> دستاویزات: Agent SDK | Hooks | Subagents | Sessions

3.1 Agentic Loop

Agentic loop میں Claude صرف جواب نہیں دیتا، بلکہ actions کی sequence چلاتا ہے:

1. Claude کو tools ساتھ کے request بھیجیں
2. Response لیں
3. `stop_reason` چیک کریں:
   - `"tool_use"` → tool چلائیں، result history میں ڈالیں، پھر دوبارہ request
   - `"end_turn"` → task مکمل، نتیجہ user کو دیں
4. مکمل ہونے تک دہرائیں

درست signal: completion لیے کہ `stop_reason == "end_turn"` دیکھیں Text parsing یا arbitrary iteration limit قابلِ اعتماد نہیں ہیں

3.2 `AgentDefinition`

```
agent = AgentDefinition(
        name="customer_support",
        description="Handles customer requests for returns and order issues",
        system_prompt="You are a customer support agent...",
        allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
)
```

اہم چیزیں: `name`، `description`، `system_prompt`، اور `allowed_tools` ہے Least privilege اصول اپنائیں

3.3 Coordinator اور Subagents

Multi-agent systems اکثر hub-and-spoke topology میں استعمال کرتی ہے:

```
                Coordinator
              /     |      \
    Subagent1  Subagent2  Subagent3
```

Coordinator task کو توڑتا، subagents کو منتخب کرتا، کام سونپتا، نتائج جوڑتا، errors handle کرتا اور، user تک final جواب پہنچاتا

اصول اہم: subagents کا context وہ؛ ہوتا الگ parent history خود بخود inherit نہیں کرتے

3.4 `Task` Tool

Subagents `Task` tool کے ذریعے spawn کیے جاتے ہیں Coordinator کے `allowed_tools` میں `"Task"` ہونا چاہیے

صحیح context passing:

- `Task: "Analyze the document"` کافی نہیں
- `Task: "Analyze this document. Full text: ... Output schema: ..."` درست ہے

Coordinator ایک response میں multiple `Task`s بھیج سکتا ہے، لہٰذا parallel subagents ممکن ہیں

3.5 Hooks

Hooks lifecycle مخصوص کے points پر work کرتے ہیں

PostToolUse tool result کو model تک پہنچانے سے پہلے normalize کر سکتا ہے:

```
@hook("PostToolUse")
def normalize_dates(tool_result):
        return normalized_result
```

PreToolUse policy violation block کر سکتا ہے:

```
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
        if tool_call.name == "process_refund" and tool_call.args.amount > 500:
                return redirect_to_escalation(tool_call)
```

یاد رکھیں: hooks deterministic ہوتے ہیں؛ prompt instructions probabilistic ہیں ہوتی Critical business rules کے لیے hooks بہتر ہیں

باب 4: Model Context Protocol (MCP)

> دستاویزات: MCP | Tools | Resources | Servers

4.1 MCP □□ کیا

MCP ایک open protocol جو □□ external systems کو Claude تین بنیادی □ کے اس □□□ جوڑتا □ سے resource types یں□:

1. **Tools** — actions □لیے □ کے functions
2. **Resources** — context data (documentation، schemas، catalogs)
3. **Prompts** — predefined task templates

4.2 MCP Servers

MCP server ایک process جو □□ tools/resources expose □□□ کرتا Connect پر □ون□ tools discover □جاتا □و □یں اور descriptions □یں□ □وتے □ مطابق استعمال □

4.3 Configuration

Project config (`.mcp.json`) □لیے □ کے ٹیم:

```
{
    "mcpServers": {
        "github": {
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-github"],
            "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
        }
    }
}
```

Project config version control ؛□□ □وتی میں secrets environment variables □یں□ آتی □ سے User config (`~/.claude.json`) personal/experimental servers □□□ □وتی □لیے □ کے

4.4 `isError`

MCP tool errors میں `isError: true` □یں□ استعمال کریں Generic errors □ بجائے □ کے structured error واپس کریں:

```
{
    "isError": true,
    "content": {
        "errorCategory": "transient",
        "isRetryable": true,
        "message": "The service is temporarily unavailable."
    }
}
```

4.5 Resources

Resources وہ data ہیں جو agent بغیر action کیے پڑھ سکتا ہے: schemas، docs، content catalogs، summaries وغیرہ۔ یہ exploratory tool calls میں کمی کرتے ہیں

باب 5: Claude Code — Configuration اور Workflows

> دستاویزات: Claude Code | Memory / CLAUDE.md | Skills | MCP | Hooks | Sub-agents | GitHub Actions | Headless

5.1 CLAUDE.md Hierarchy

CLAUDE.md تین سطحوں پر ہو سکتا ہے:

- **User-level:** `~/.claude/CLAUDE.md` — صرف اسی user پر لاگو
- **Project-level:** `.claude/CLAUDE.md` یا root `CLAUDE.md` — پوری team کے لیے
- **Directory-level:** subdirectories میں CLAUDE.md — مخصوص folder conventions

5.2 `@path` Syntax

CLAUDE.md میں `@path` سے external files refer کیے جا سکتے ہیں تاکہ config modular ہو:

```
Coding standards @./standards/coding-style.md میں
Test rules @./standards/testing-requirements.md میں
```

5.3 `.claude/rules/`

`.claude/rules/` rules کو topic کے لحاظ سے organize کرتا ہے اور `paths` کے ذریعے conditional loading دیتا ہے:

```
---
paths: ["src/api/**/*"]
---

API files کے لیے async/await اور explicit error handling استعمال کریں
```

5.4 Commands اور Skills

`.claude/commands/` project commands □جبکہ، □□ لیے □کا `~/.claude/commands/` personal commands □□لیے □کا `.claude/skills/` میں frontmatter ساتھ □کا skills □ہیں ملتی

5.5 Skills اور `context: fork`

`context: fork` skill کو isolated context □□□ میں چلاتا `allowed-tools` □سے tool access اور، محدود کریں `argument-hint` □سے required input □مانگیں

5.6 Planning Mode

Planning mode بڑی architectural تبدیلیوں، multiple approaches، اور unfamiliar codebases □کے لیے بہ□ترین □□□ Direct execution اور واضح □چھوٹے changes □□□ بہ□تر لیے □کے

5.7 `/compact` اور `/memory`

`/compact` context compress اور، □□ کرتا `/memory` notes اور preferences □□□ سنبھالنے □میں مدد دیتا

5.8 Claude Code CLI for CI/CD

`-p` / `--print` non-interactive mode □□ ہوتا□ استعمال لیے □کے:

```
claude -p "Analyze this pull request for security issues"
```

Structured output □لیے □کے `--output-format json` اور `--json-schema` □کریں استعمال

---

6 باب: Prompt Engineering — Advanced Techniques

> دستاویزات: Prompt Engineering | Anthropic Cookbook

6.1 Few-shot Prompting

Few-shot prompting 4–2 میں examples کر □د expected behavior □یہ □□□ دکھایا جانا vague instructions □سے □کیونکہ □□ مؤثر □زیادہ example ambiguity □□□ کم کرتا

6.2 Explicit Criteria

Vague instructions □بجائ□ کا clear criteria کیا :دیں flag کیا، □□ کرنا ignore اور، □□ کرنا severity □کیس decide کرنی □□□

6.3 Prompt Chaining

Complex task کو focused steps میں توڑیں: local analysis، پھر integration pass□ی□ □ attention dilution کم کرتا □□□

6.4 Interview Pattern

Claude □س clarifying questions پوچھوانا design ambiguity جب خاص طور پر، □□ لاتا سامن□ domain unfamiliar □و□

6.5 Validation and Retry

Extraction fail □و تو original document، incorrect output، اور specific validation error ساتھ □کا retry □کریں

6.6 Self-correction

Model stated value اور calculated value ؛□نکال دونوں conflict □و تو discrepancy handle □کریں

---

7 باب: Message Batches API

> دستاویزات: Message Batches

7.1 Overview

Message Batches API asynchronous processing 50 :□□ □لی□ کا% savings، up to 24 hours window، اور multi-turn tool calling supported □نہیں□ `custom_id` □س requests اور responses link □میں□ □وتا□

7.2 کب استعمال کریں

Pre-merge checks اور interactive tasks □لی□ کا synchronous API، □جبک overnight reports اور bulk jobs □کا □لی□ Batch API □استعمال کریں

7.3 `custom_id`

`custom_id` result کو original document □سے جوڑنا□، failures identify کرنا□، اور صرف failed items □دوبارہ submit □□□ کرنا□ میں مدد دیتا

---

8 باب: Task Decomposition Strategies

8.1 Fixed Pipelines

Predictable tasks □لیے□ کے fixed pipeline □جیسے□ استعمال کریں، Document → Extraction → Validation → Final output□

8.2 Dynamic Decomposition

Open-ended tasks میں subtasks intermediate results □پر□ کی بنیاد پر بنتے□ □ہیں□

8.3 Multi-pass Code Review

Large PRs □لیے□ کے per-file analysis بعد کے cross-file integration pass □تاکہ کریں attention dilution □ہو□ کم

---

9 باب: Escalation اور Human-in-the-Loop

9.1 کب Escalate کریں

Explicit user request، policy gaps، progress ہونا□ □نہ، threshold اوپر سے□ financial action، یا ambiguous matching کی صورت میں escalate □کریں Sentiment analysis اور self-rated confidence unreliable □ہیں□

9.2 Escalation Patterns

□مسئلہ □□پر resolve اگر پھر، کریں کوشش کی کرنا□ policy gap یا explicit insistence □ہو تو human کو hand off کریں□

9.3 Structured Handoff

Escalation پر customer ID، issue summary، actions taken، اور recommended action □تاکہ شامل کریں human operator مکمل کو context □□ملے

9.4 Confidence Calibration

Field-level confidence scores، calibration on labeled data، اور stratified sampling استعمال کریں۔

باب 10: Multi-agent Systems میں Error Handling

10.1 Error Categories

Transient errors retryable ہیں۔، validation errors input fix ہیں۔ مانگتی، business errors policy-driven ہوتی۔ ہیں، اور permission errors generally escalate ہیں۔ کی جاتی

10.2 Anti-patterns

Generic “search unavailable” response، silent suppression، یا پورا workflow abort ۔۔۔ ۔ کرنا نقصان د Structured errors ۔ واپس کریں

10.3 Structured Subagent Error

Structured error میں failure type، attempted query، partial results، alternative approaches، اور coverage impact ۔ شامل کریں

10.4 Final Synthesis میں Coverage

Report سی کون ۔ کہ کریں واضح میں sections fully covered سی کون ہیں، ۔ partially covered ہیں، اور کہاں gaps ۔ ہیں باقی

باب 11: Production Systems میں Context Management

11.1 Facts Separate Block میں

Transactional facts کو persistent case facts block ۔ تاکہ رکھیں میں summarization باوجود ۔ کہ critical data ضائع ۔ ہو ۔ نہ

11.2 Tool Results Trimming

PostToolUse hook صرف ۔ سہ relevant fields ۔ رکھیں

11.3 Position-aware Input

Important findings شروع میں اور action items آخر میں رکھیں تاکہ lost-in-the-middle effect کم ہو۔

11.4 Scratchpad Files

Verbose findings scratchpad میں لکھیں تاکہ long investigations میں context محفوظ رہے۔

11.5 Subagents کو Delegate کرنا

Exploration subagents میں دیں اور main context میں صرف summary رکھیں۔

11.6 Structured State Persistence

ہر agent اپنی state manifest یا JSON file میں export کرے تاکہ crash recovery ممکن ہو۔

باب 12: Provenance کو محفوظ رکھنا

12.1 Attribution Loss

Claim کے ساتھ source URL، publication date، اور confidence رکھیں۔

12.2 Conflicting Data

اگر sources مختلف values دیں تو دونوں کو attribution کے ساتھ محفوظ رکھیں اور conflict flag کریں۔

12.3 Dates

Temporal differences کو contradiction سمجھنے سے پہلے publication dates دیکھیں۔

12.4 Content Type کے مطابق Render

Financial data tables میں، news prose میں، technical findings structured lists اور time series میں chronological order میں دکھائیں۔

13 باب: Claude Code کے Built-in Tools

13.1 Tool Selection Reference

Task	Tool	Example
pattern سے file تلاش کرنا	**Glob**	`**/*.test.tsx`
file contents میں search	**Grep**	function name, error message
file پڑھنا	**Read**	analysis کے لیے
نیا file بنانا	**Write**	scratch سے
precise edit	**Edit**	unique text match
shell command	**Bash**	git، npm، tests

13.2 Incremental Investigation

ایک ساتھ سب files کھولیں؛ پڑھیں نہ entry points، پھر usages، پھر consumer files دیکھیں

13.3 Read + Write Fallback

اگر Edit fail ہو تو Read کریں، تبدیلی programmatically کریں، پھر Write کریں

حصہ II: امتحانی ڈومین نوٹس

ڈومین 1: Agent Architecture اور Orchestration (27%)

Agentic loops، coordinator–subagent architecture، `Task` کے ذریعے spawning، hooks، escalation، اور multi-step workflows پر فوکس کریں

ڈومین 2: Tool Design اور MCP Integration (18%)

Tool descriptions، `tool_choice`, JSON schema output، structured errors، MCP servers، resources، اور built-in tools کے فرق کو سمجھیں

ڈومین 3: Claude Code Configuration اور Workflows (20%)

CLAUDE.md hierarchy، `.claude/rules/`، commands، skills، planning mode، `context: fork`، اور CI/CD integration میں ہم

ڈومین 4: Prompt Engineering اور Structured Output (20%)

Few-shot prompting، explicit criteria، prompt chaining، validation/retry loops، JSON schema enforcement، اور batch-friendly prompting پر توجہ دیں

ڈومین 5: Context Management اور Reliability (15%)

Summarization risks، lost-in-the-middle، trimming tool output، confidence calibration، escalation، اور provenance preservation یاد رکھیں

Scenario 7 مشق سوالات کا: Conversational AI Architecture Patterns

سوال 61

صورتحال: آپ کا `remove_team_member` tool میں execution کے اثرات کا پتا لگنے سے پہلے preview کرنے کا لیے برا راست agent سے معلوم ہوتا ہے کہ Production monitoring میں `dry_run: boolean` parameter کرتا ہے آپ کو یہ یقینی بنانا ہے کہ bypass کو preview step کر کے call کا ساتھ `dry_run=false` deletion ایک سے پتا لگنے سے preview جسے ہو user نے explicitly confirm کیا ہو

کونسی approach سب سے قابل اعتماد ہے؟

- A) Server-side validation شامل کریں جو صرف اسی وقت allow 60 کرے جب پچھلے `dry_run=false` call کے ساتھ parameters میں یکساں seconds `dry_run=true` call ہوئی ہو
- B) Tool کو confirmation کی ضرورت کے طور پر annotate کریں اور orchestration layer کو configure کریں کہ calls forward کرنے سے پہلے user کی approval مانگے annotated tools کو
- C) Tool description میں تفصیلی instructions اور few-shot examples شامل کریں جن میں agent سے مطالبہ کرے کہ وہ ہمیشہ پہلے `dry_run=true` call کرے
- D) دو tools سے replace کریں: `preview_remove_member` impact details اور ایک single-use confirmation token واپس کرتا ہے؛ `execute_remove_member` کو وہ token درکار ہے، execution کو preview سے bind کرتا ہے **[درست]**

کیوں D: Token-binding approach یہ سے architecturally پتا لگتا ہے کہ بغیر preview ناممکن ہو جاتا ہے کہ preview tool ہی generate کر سکتا ہے جو صرف token ہو اور execute tool کو literally execution ہو — LLM instructions کی پابندی (C)، timing heuristics (A)، یا orchestration infrastructure جو approach یہ واحد ہے (B) پر انحصار کے بجائے code level پر constraint enforce کرتی ہے

سوال 62

صورتحال: Production monitoring سے معلوم ہوتا ہے کہ آپ کا `search_catalog` tool 12% وقت fail ہوتا ہے جو کبھی 8% query syntax errors پر کامیاب ہوتے ہیں، اور 4 retry میں جو network timeouts ہیں: 90% فی الحال دونوں ہوتے ہیں، جس سے بچ فائدہ return یکساں طریقہ سے error types کامیاب نہیں ہوتے retries ہوتی ہیں۔

آپ tool کی error handling کیسے modify کریں گے؟

- A) System prompt میں few-shot examples شامل کریں جو demonstrate کریں کہ network errors اور syntax errors میں فرق کیسے کریں
- B) تمام errors پر uniformly exponential backoff retry logic apply کریں
- C) Tool کے اندر network timeouts کے لیے automatic retry with backoff implement کریں؛ syntax errors کو parameter validation details **[درست]** کے ساتھ فوری واپس کریں
- D) تمام errors کو `retryable` boolean flag اور error type details کے ساتھ واپس کریں

کیوں C: Tool level پر transient errors کے لیے retries handle کرنا صحیح abstraction boundary ہے — tool کو agent پر deterministic retry logic implement کر بغیر سکتا ہے کہ بار میں یقینی علم ہے اور وہ error type انحصار کہ کہ ہو flag (D) interpret یا prompt instructions (A) follow کرے Uniform backoff (B) syntax errors پر وقت ضائع کرتا ہے جو کبھی کامیاب نہیں ہوں گے

سوال 63

صورتحال: Investment strategy کی گفتگو کے کئی turns میں، user "میری risk tolerance اور بہت کم ہے" کہتا ہے "کرنا چاہیے؟ invest کرنا چاہتا ہوں" اب وہ پوچھتا ہے: "مجھے کس چیز میں returns maximize پھر "میں اپنے

کونسا approach کے سب سے بہتر طریقے سے یقینی بناتا ہے کہ recommendation user کی اصل priority کے ساتھ آگے بڑھیں ہم آہنگ ہو؟

- A) user تضاد کو سامنے لائیں اور **[درست]** سے پوچھیں کہ کونسی چیز زیادہ اہم ہے
- B) دونوں scenarios الگ الگ کے لیے recommendations فراہم کریں
- C) سب سے preference حال میں بیان کی گئی کے ساتھ آگے بڑھیں
- D) تضاد کو حل کیے بغیر balanced portfolio تجویز کریں

کیوں A: جب user کی preferences اور clarification براہ راست متضاد ہوں، تو تضاد سامنے لانا اور مانگنا ہی واحد طریقہ ہے کہ recommendation user کا اصل ارادہ سے ہم آہنگ ہو Returns maximize اور low risk tolerance بنیادی طور پر ناقابل مطابقت اہداف ہیں جن کے لیے انسانی فیصلہ درکار ہے

سوال 64

صورتحال: Users کئی conversation turns میں playlist preferences refine کرتے ہیں User "مجھے jazz پسند ہیں؟ genres پوچھتا ہے "آپ کو کون سے Claude، messages بعد پسند ہے" کہنے کے دو

سب سے زیادہ ممکنہ وجہ کیا ہے؟

- A) Claude کو conversation memory برقرار رکھنے کے لیے vector database connection درکار

- B) Model کی context window exceed ہو گئی ہے
- C) Claude API کو `session_id` parameter درکار ہے
- D) آپ کی application `messages` array میں پچھلے messages **[درست]** شامل نہیں کر رہی ہے

D کیوں: Claude کا server-side memory نہیں ہے — ہر API call stateless ہے اور ہر request کا `messages` array میں مکمل conversation history شامل کیے بغیر، Claude کا پاس پچھلے turns کا کوئی علم نہیں ہوتا۔ Vector databases (A) اور `session_id` (C) Claude کے architecture کا حصہ نہیں ہیں؛ دو ناممکن ہیں (B) context window overflow کا لیے messages کے exchange

---

سوال 65

صورتحال: 40 منٹ کے cooking session کے بعد، conversation 78,000 tokens تک پہنچ گئی ہے۔ History میں allergies، recipe scaling، clarified cooking terms، اور عمومی discussion کے اہم معلومات شامل ہیں۔ آپ کو محفوظ رکھنے ہوئے tokens کم کرنے ہیں۔

کونسا token reduction اور approach preservation کے درمیان بہترین توازن رکھتا ہے؟

- A) پوری conversation history کا خلاصہ کریں
- B) 20,000 صرف سب سے حالیہ tokens رکھیں
- C) اہم structured data (allergies، quantities، preferences) عمومی discussion نکالیں، اور کا خلاصہ کریں، حالیہ exchanges کو verbatim **[درست]** رکھیں
- D) مکمل conversation externally store کریں اور semantic search کے ذریعے متعلقہ حصے retrieve کریں

C کیوں: Hybrid approach سب سے cost کم پر سب سے valuable معلومات محفوظ رکھتی ہے۔ Allergies اور recipe quantities جیسے critical facts ایک compact structured block میں extract ہوتے ہیں (summarization عمومی discussion summarize کرنے سے precision loss ہوئے بچاتے ہوئے)، اور حالیہ exchanges verbatim رکھنے ہوتی ہیں۔ Options A اور B اہم dietary information کھو سکتے ہیں اور verbatim کا لیے conversational coherence ضرورت سے زیادہ پیچیدہ ہے کے لیے cooking session ایک D ہے؛

سوال 66

صورتحال: Users طویل conversations میں assistant کے پہلے topics اور preferences رپورٹ کرتے ہیں کے message pairs صرف آخری 25 implementation رکھتی ہے۔ آپ کی موجودہ حساب کھو دیتا ہے

کیا solution سب سے مؤثر ہے؟

- A) Hybrid approach: پرانے messages کا خلاصہ کرتے messages verbatim **[درست]** حالیہ ہوئے رکھیں
- B) مکمل conversation history پر vector similarity search
- C) Window کو 50 message pairs تک بڑھائیں
- D) ہر turn میں dropped messages کا خلاصہ کریں اور running summary آگے جوڑیں

A کیوں: Hybrid approach مسئلے کے دونوں پہلوؤں کو حل کرتی ہے: exact recent context محفوظ رکھتی ہے جبکہ پہلے کی preferences کی compressed representation برقرار رکھتی ہے (conversational coherence کے لیے اہم context) Vector search (B) صرف اسی مسئلے کو ملتوی کرتا ہے (C) Window بڑھانا semantically سے current query کر سکتی ہے جو مشابہ نہ ہو

سوال 67

صورتحال: Users جب کہ conversations 50 turns رپورٹ کرتے ہیں تو ہر response سے پہلے latency بڑھ جاتی ہے اور costs بھی۔

بنیادی وجہ کیا ہے؟

- A) ہر API request کے ساتھ پوری conversation history **[درست]** شامل ہوتی ہے
- B) Model gradually لمبے responses generate کرتا ہے
- C) History بڑھنے کے ساتھ database operations سست ہو جاتے ہیں
- D) Model زیادہ processing والا internal user profile بناتا ہے

کیوں A: Claude کی API مکمل طور پر stateless ہے — ہر request کو `messages` array میں پوری conversation history شامل کرنی ہوتی ہے، جیسے جیسے conversations بڑھتی ہیں، ہر request زیادہ tokens، جو Model calls کے درمیان کوئی directly processing latency اور cost دونوں بڑھاتا جاتی ہے، internal state رکھتا نہیں (D غلط ہے)۔

سوال 68

صورتحال: تین مہینوں کے weekly sessions بعد، conversation history 85,000 tokens جب تک بڑھ گئی ہے، پچھلی discussions کے assistant کے بارے میں کیا نتیجہ اخذ کیا تھا؟"، تو user کے theme کو پوچھتا ہے "ہم نے

کیا approach سب سے مؤثر ہے؟

- A) Rolling window truncation
- B) Key conclusions capture کرتے ہوئے progressive summarization
- C) Relevant exchanges retrieve کرنے کے لیے semantic embeddings **[درست]**
- D) Discussion conclusions mark کرنے کے لیے structured XML tags شامل کریں

کیوں C: Conversation history پر semantic search واحد approach ہے جو تین مہینوں کی discussion کو scale پر specific relevant exchanges surface کر سکتی ہے جبکہ Rolling window truncation (A) history کا بیشتر حصہ ضائع کر دے گی Progressive summarization (B) discussions کو abstractions میں compress کرتی ہے، specific conclusions جن کے بارے میں users کھو دیتی ہے پوچھتے ہیں۔

سوال 69

صورتحال: QA testing کے دوران، Claude 15–10 turns پہلے system prompt guidelines follow کرتا ہے، لیکن بعد کے responses میں deviation آ جاتی ہے Conversation ابھی token limits کے اندر ہے۔

بہترین solution کیا ہے؟

- A) Behavioral guidelines کو پہلے user message میں منتقل کریں
- B) 20 turns بعد نئی conversation شروع کریں
- C) Conversation breakpoints پر guidelines reinforce کرنے والے user-role messages insert **[درست]** کریں
- D) Non-compliant responses کو regenerate کرنے کے لیے post-response validation استعمال کریں

C کیوں: Behavioral reminders کا periodic injection instruction drift کا براہ راست مقابلہ کرتا ہے، history accumulate ہونے کے ساتھ regular intervals پر constraints re-establish کر کے Guidelines کو پورا user message میں منتقل کرنا (A) ان کا authority کم کرتا ہے Post-response validation (D) corrective ہے نہ کہ preventive اور significant latency شامل کرتی ہے

سوال 70

صورتحال: آپ کے AI tutor میں 2,800-token system prompt جو teaching methodology اور adaptation rules کو نظرانداز کرنا شروع کر دیتا ہے، assistant proficiency levels بعد کے turns کرتا ہے define 12

کیا fix سب سے مؤثر ہے؟

- A) 4–5 turns کے بعد reminders inject کریں
- B) Verbose rules کو few-shot examples سے replace کریں جو proficiency-level adaptation demonstrate کرنے والا ہے **[درست]**
- C) Critical rules کو system prompt کے آخر میں رکھیں
- D) Responses evaluate کریں اور اگر difficulty level match نہ ہو تو regenerate کریں

B کیوں: Declarative rules 2,800-token system prompt drift کا شکار ہے کیونکہ abstract rules کو concrete few-shot examples سے replace کرنا جو correct proficiency-level adaptation demonstrate کریں، model کو match کرنے کا clear behavioral patterns کئی ہے — ہے دیتا turns میں abstract instructions سے زیادہ reliably follow ہوتا ہے

سوال 71

صورتحال: آپ کے assistant کو enthusiastic tone برقرار رکھنا، اپنی reasoning explain کرنا، اور clarifying questions پوچھنے ہیں ہے behavioral guidelines کو define؟ ہونی چاہئیں

یہ behavioral guidelines کو define؟ ہونی چاہئیں

- A) ہر user message کے پہلے سے prepend کریں
- B) System prompt میں **[درست]**
- C) پہلے assistant message میں
- D) Environment variables میں

B کیوں: System prompt خاص طور پر پوری conversation میں apply ہونے والی persistent behavioral constraints اور guidelines ہر user message کے لیے ڈیزائن کیا گیا ہے (A) redundant سے پہلے شامل کرنا overhead پہلا assistant message (C) unreliable ہے کیونکہ model اپنے پہلے statements سے deviate کر سکتا ہے Environment variables (D) model کے behavior پر اثر نہیں ڈالتے

سوال 72

صورتحال: Users repetitive response openings جیسے "یقیناً!" اور "میں مدد کرنے میں خوش ہوں!" رپورٹ کرتے ہیں

کیا □□؟ approach سب سے مؤثر

- A) Direct response opening کے ساتھ partial assistant message append **[درست]** کریں□
- B) Temperature setting کم کریں□
- C) Greetings □لیے کے □ٹانے responses کو post-process کریں□
- D) System prompt میں ان phrases کی بچنے سے instructions شامل کریں□

A کیوں: Assistant کے response کو direct جواب کی شروعات سے □ prefill کرنا generation level پر greeting patterns □□ روکتا — model prefill سے continue □ بجائے نئے □□ کرتا opening phrases generate □□ کرنے کے System prompt instructions (D) مدد کر سکتی □یں لیکن کم reliable □یں□ Post-processing (C) ایک fragile workaround □□□ Temperature (B) randomness control □□ کرتا، specific phrase patterns □نہیں

سوال 73

صورتحال: ایک webhook □ آپ کے system کو notify □ کے □□ کرتا user کا package ship □ جبکہ □و گیا □□ user actively chat □ کے □یں □تے □□□ آپ چا□ رہا □ assistant اسے اگلا response میں naturally □□ شامل کر

کیا □□؟ approach بہ□ترین

- A) Shipping status کو system prompt □ میں شامل کریں□
- B) فوری synthetic user message □ بھیجیں□
- C) □ر turn میں assistant کو status tool call □ کرنے پر مجبور کریں□
- D) Status update □ کو اگلا user message □ کے prefix پر □ طور پر append **[درست]** کریں□

D کیوں: Status update □ کو اگلا user message □ کے prefix کرنا شامل طور پر □ کے flow کو interrupt بغیر کیے □ natural conversation boundary پر real-time context inject □□□ کرتا System prompt modify کرنا (A) session rebuild □□□ کی ضرورت Synthetic user message (B) natural dialogue flow کو disrupt ر□ □□□ کر سکتا turn میں tool call force کرنا (C) اس وقت wasteful جب □□ events rare □وں□

سوال 74

صورتحال: Users □ لیے کے "پارٹی اکثر venue book □ جیسے "کریں requests □یں□ بھیجتے Assistant 4+ clarifying questions 35 □ جس سے □□، پوچھتا% abandonment □□□ □وتا

کونسا approach trade-off سے □ بہ□ترین طریقہ□ کو improve کرتا □□؟

- A) Hidden defaults □ کے ساتھ آگے□ بڑھیں□
- B) تمام clarifying questions ایک compound message □ میں پوچھیں□
- C) Assumptions explicitly بیان کریں اور corrections **[درست]** □ مدعو کرتے □وئے آگے□ بڑھیں□
- D) Structured intake form □ استعمال کریں□

C کیوں: Assumptions explicitly بیان کرنا اور آگے□ بڑھنا user کو فوری، مفید response کو غلط □□ دیتا جبکہ □ کو نہ□یں بتاتے □ کہ کیا Hidden defaults (A) user کرنے کی صلاحیت برقرار رکھتا □□□ کو correct assumptions کا تقاضا کرتی □□□ upfront effort سے user پھر بھی Compound question list (B) کیا گیا assume □ Structured form (D) کم friction □□□ شامل کرتا زیادہ□ کی بجائے

سوال 75

صورتحال: آپ کا assistant contractor-persona system prompt ابتدائی □□□ استعمال کرتا turns rules follow کرتا □ہیں، لیکن turn 7 تک assistant generic advice □□□ دیتا Conversation length 2,500 صرف tokens □□□

سب سے زیادہ ممکنہ وجہ کیا □□؟

- A) System prompts صرف initial behavior establish □کرتے □ہیں
- B) Turns accumulate ساتھ کے □ہونے model کی attention □□□ کمزور □ہوتی
- C) Accumulated assistant responses system prompt کے اثر کو dilute **[درست]** □کرتے □ہیں
- D) System prompt □□□ صرف ایک بار بھیجا جاتا

C کیوں: جیسے جیسے conversation history میں assistant responses □جمع □ہوتے □ہیں، system prompt کے assistant-generated content کا proportion بڑھتے □ہوئے □کرنے والا text behavioral constraints کو reflect کم □ہوتا جاتا □□□ Model چھوٹے token lengths پر بھی system prompt instructions کے بجائے اپنے □پہلے outputs کے patterns کو زیادہ follow □□ کرتا، drift کو compound □□□ کرتا

سوال 76

صورتحال: Users مبہم requests کرتے □ہیں جیسے "کیا آپ میں مدد کر سکتے □ہیں؟ کیا آپ Assistant کئی سوالات پوچھ کر respond □□ کرتا (کون سی report؟ کیا مدد؟ deadline ؟ کیا □□؟)، 40 جس سے% abandonment □□□ ہوتا□

بہترین□ solution کیا □□؟

- A) Reasonable assumptions بیان کریں، اور explicitly کریں، انہیں adjustments **[درست]** کی پیشکش کریں□
- B) Respond □پہلے سے کرنے smaller model کے ساتھ ambiguity classify □کریں
- C) Assumptions بغیر بیان کیے predefined interpretations □استعمال کریں
- D) Assistant کو □ہر turn میں ایک clarifying question □تک محدود کریں

A کیوں: Reasonable stated assumptions بڑھنا آگے ساتھ کے user کو informed اور in control □رکھتے □ہوئے back-and-forth □□□ کو مکمل طور پر ختم کرتا Silent predefined interpretations (C) users کو confuse کرتی ہیں جب □response سے ارادے کے ان match □ہو □نہ One-question limit (D) پھر بھی back-and-forth turns کی ضرورت □□□ Smaller classification model (B) core UX بغیر مسئلے □حل کیے latency اور infrastructure complexity □□□ شامل کرتا

عملی مشقیں

Exercise 1: Multi-tool Agent with Escalation Logic

Goal: tool integration، structured errors، اور escalation ساتھ کے agent loop □بنائیں

Exercise 2: Configuring Claude Code for Team Development

Goal: CLAUDE.md، custom commands، path-specific rules، اور MCP servers configure کریں۔

Exercise 3: Structured Data Extraction Pipeline

Goal: JSON schema، validation/retry loops، اور Message Batches API ساتھ کے extraction pipeline بنائیں۔

Exercise 4: Designing and Debugging a Multi-agent Research Pipeline

Goal: subagent orchestration، explicit context passing، error propagation، اور source tracking implement کریں۔

Appendix: Technologies and Concepts

Technology	Key aspects
Claude Agent SDK	agent loops, `stop_reason`, hooks, `Task`, `allowedTools`
MCP	servers, tools, resources, `isError`, `.mcp.json`
Claude Code	CLAUDE.md, `.claude/rules/`, `.claude/commands/`, `.claude/skills/`, planning mode
Claude Code CLI	`-p`, `--output-format json`, `--json-schema`
Claude API	`tool_use`, `tool_choice`, `stop_reason`, `max_tokens`
Message Batches API	50% savings, 24-hour window, `custom_id`
JSON Schema	required/optional, nullable, enum values
Few-shot prompting	ambiguous scenarios کے لیے examples
Prompt chaining	sequential decomposition
Context window	token budget, lost-in-the-middle
Confidence calibration	field-level scoring, stratified sampling

---

Out-of-Scope Topics

یہ موضوعات امتحان میں شامل نہیں ہیں:

- Claude models کی fine-tuning
- Authentication، billing، یا account management
- مخصوص programming languages کی تفصیلی implementation
- MCP servers کی hosting/infrastructure
- Claude کی internal architecture یا model weights
- Constitutional AI، RLHF، یا safety training
- Embeddings یا vector databases کی implementation details
- Computer use / browser automation
- Vision / image analysis
- Streaming API یا SSE
- Rate limiting اور detailed API cost calculations
- OAuth اور API key rotation details
- Cloud-provider-specific configs
- Performance benchmarks
- Prompt caching implementation details
- Token counting algorithms

Preparation Recommendations

1. Claude Agent SDK ساتھ ایک □ کا agent بنائیں
2. Claude Code کو real project پر configure کریں
3. MCP tools design اور test کریں
4. Data extraction pipeline بنائیں
5. Prompt engineering practice کریں
6. Context management patterns سیکھیں
7. Escalation اور human-in-the-loop workflows سمجھیں
8. Practice exam حل کریں